DELTA-V TYCOON

Team Brief 1 of 6 · Version 1.1 (13 Nov 2026)

Team Lead

Game design owner · science content & all player-facing text · pitch, video, slides · submission

NASA Space Apps Cairo 2026 · Challenge: Space Mission Design Game
Hackathon: Fri 13 Nov 1:00 PM → Sat 14 Nov 1:00 PM (Cairo time)

This brief contains	Part B: your role, tasks, order of work and acceptance criteria. Part A: the shared project bible (identical in all six briefs): game design, every data value, every formula, file names and code contracts.
How to use it	Read Part B first, then Part A. Keep Part A open while you work. If Part A and Part B disagree, Part A wins. The brief is self-contained: hand both parts to any collaborator or tool and the work can be executed without extra context.
You own	src/data/*.json content, README.md, docs/GDD.md, docs/SOURCES.md, the demo video, the slides, the submission.
The six briefs	1 Team Lead · 2 Programmer A (core & rooms) · 3 Programmer B (UI & build) · 4 Character Artist · 5 Environment & Asset Artist · 6 QA & Playtest. Together they produce one game: every file in the repository has exactly one owner.
Golden rule	No production work before Friday 1:00 PM. Feature freeze Saturday 3:00 AM. Submit by Saturday 12:00 PM.

Contents

PART B — TEAM LEAD: GAME DESIGN, SCIENCE CONTENT, PITCH & SUBMISSION 3

B1. Your mission . . . 3
B2. Deliverables and deadlines . . . 3
B3. Before H0 (allowed: setup only) . . . 3
B4. Step by step . . . 3
B5. Your daily rhythm during the event . . . 16

PART A — SHARED PROJECT BIBLE (identical in all six briefs) . . . 17

A0. What changed in v1.1 (read this first) . . . 17
A1. How to use this document . . . 17
A2. Event facts and hard rules . . . 17
A3. The challenge, and how we answer every part of it . . . 18
A4. Game concept . . . 19
A5. Feature list (IDs, priority, owner) . . . 20
A6. Screens, routes and flow . . . 21
A7. Game roles and team mechanics . . . 21
A8. Game data (exact values) . . . 22
A9. Formulas (Programmer A implements exactly; QA verifies with A17) . . . 27
A10. Readiness checks (runChecks) . . . 29
A11. Mission simulation (createSimulation) . . . 30
A12. Scoring (scoreMission) — maximum 1000 . . . 30
A13. Online rooms (Firebase Realtime Database) . . . 31
A14. Tech stack, repository and code contracts . . . 32
A15. Art direction and the complete asset manifest . . . 36
A16. Timeline, communication and definition of done . . . 38
A17. Reference designs (expected values — tests and QA use these) . . . 39

PART B — TEAM LEAD: GAME DESIGN, SCIENCE CONTENT, PITCH & SUBMISSION

B1. Your mission

You are the **owner of Part A**, the **author of every word the player reads**, the **guardian of scientific validity**, the **producer** who keeps six people on schedule, and the **only person who submits**. You do not write game code. You write the game's data and text (`src/data/*.json`), the docs (`README.md`, `docs/GDD.md`, `docs/SOURCES.md`), the 30-second video and the 7 slides, and you run the submission.

You own: `src/data/*.json` (content; types come from Programmer A), `README.md`, `LICENSE`, `docs/GDD.md`, `docs/SOURCES.md`, the demo video, the slides, the project summary, both submissions, `#bible-changes`.

B2. Deliverables and deadlines

By	Deliverable
Before H0	Empty public repo `delta-v-tycoon` created; Pages source set to GitHub Actions; all members added as collaborators; team registered on the global site with every member invited; Discord channels created (A16)
H0:30	Kickoff done; the four official NASA resources (A3) added to `docs/SOURCES.md`
H2	All JSON files v1.1 pushed: constants, orbits, launchVehicles, components, instruments, scenarios (with userSegment + twins), difficulties, events, checks, glossary, dialog, characters
H3	`docs/GDD.md` = short version of A4–A7 + link to this brief
H6	Review first build; first balance notes
H9	Text polish pass on every screen (play the game, fix wording in JSON)
H12 (1 AM)	Online go/no-go decision with QA (cutoff rule, A16)
H13 (2 AM)	Balance pass using Programmer A's `scripts/balance.ts` table
H14 (3 AM)	**Feature freeze call.** Collect footage with QA
H16–H21 (5–10 AM)	Video uploaded (public), slides exported, README final, summary final
H22–H23	Submit local form → global site; QA verifies links
~5 PM	People's Choice live pitch ready (2 minutes + live demo)

B3. Before H0 (allowed: setup only)

1. Create the **empty** public GitHub repo `delta-v-tycoon` (README only). Settings → Pages → Source = GitHub Actions. Add all members as collaborators.
2. On the global NASA Space Apps site: confirm the team, Cairo as location, every member invited and accepted, the **Space Mission Design Game** challenge selected.
3. Create a team Discord (or private channels) with `#general`, `#bible-changes`, `#builds`, `#bugs`, `#art-drops`, `#help`. Pin Part A.
4. Make sure everyone knows: arrive 8:00 AM with national ID, laptop, charger, extension cord; one member (QA recommended) is virtual on day 1.
5. Prepare (but do not produce): a YouTube account for an unlisted/public upload, a Google Slides account, a screen recorder (OBS Studio), a simple video editor (CapCut desktop, DaVinci Resolve, or Shotcut).

B4. Step by step

Step 1 (H0–H0:30): kickoff (script)

1. "We build Delta-V Tycoon. Part A is the law; if it is not in Part A, ask me."
2. Walk through A4 (loop), A5 (features and priorities), A16 (timeline and cutoff rule), A15 (palette).
3. Confirm owners and folders (A14). Confirm Discord status pings every 2 hours.

4. Walk through the four official resources and where each lives in the game (A3 table). Everyone should be able to say in one sentence how we used CMLs and the mission segments; judges may ask anyone.
5. Start the clock. Programmer B scaffolds; Programmer A writes types; artists do style tests; QA writes the test plan; you write JSON.

Step 2 (H0:30–H2): write all data files

Write each file as a JSON array or object matching Programmer A's `types.ts` (A14.1). Numbers come **only** from Part A §A8; the text comes from this section. Validate each file at jsonlint.com before pushing. Use the exact field names below.

`constants.json` — the object from A9 constants, e.g. `{ "G0": 9.81, "SOLAR_EFFICIENCY": 0.85, ... }`.

`orbits.json` — six objects. Example (complete):

```
{
  "id": "sso", "name": "Sun-Synchronous Orbit", "destination": "SSO", "altitudeKm": 700, "periodMin": 99,
  "sunFraction": 0.65, "eclipseMin": 35, "solarFactor": 1.0,
  "insertionDvMs": 0, "stationKeepingDvMsPerYear": 10, "disposalDvMs": 150, "naturalLifetimeYears": 25,
  "commsFactor": 1.0, "contactHours": { "campus": 0.4, "commercial": 3.0, "deepspace": 1.0 },
  "lightDelaySec": 0, "radiation": 1.2, "opsCostMPerYear": 2, "cruiseMonths": 0,
  "magnetorquersWork": true, "isNearEarth": true, "debrisRule": true,
  "description": "<the A8.1 explanation for sso>",
  "pros": ["Same lighting every pass — perfect for comparing images", "Covers the whole globe", "Frequent passes
      over polar ground stations"],
  "cons": ["Needs about 150 m/s to deorbit at end of life", "Eclipse every orbit — needs a battery", "Short
      contact with a single mid-latitude station"]
}
```

Pros / cons for the other orbits:

- **leo** — pros: sharp images; cheapest to reach; short radio distance. cons: short contacts (~8 min per pass); drag needs fuel to stay up; must deorbit at end of life.
- **geo** — pros: stares at one region all the time; 24-hour contact with one station; almost no eclipses. cons: far = blurry details and weaker signal; spacecraft must burn 1,500 m/s to circularize; stronger radiation.
- **lunar** — pros: maps the entire Moon including polar craters; close to future landing sites. cons: needs the deep-space network; lumpy lunar gravity costs 100 m/s per year; no magnetic field for magnetorquers.
- **l2** — pros: cold, stable, no eclipses — ideal for telescopes; Sun and Earth always behind you. cons: 1.5 million km away — deep-space network required; limited contact hours.
- **mars** — pros: global view of Mars; can relay data for rovers. cons: sunlight only 43%; radio delay up to 22 minutes; launch window only every 26 months; 7-month cruise.

`launchVehicles.json` — five objects from A8.3 (`capacityKg` as an object, e.g. `{ "LEO": 300, "SSO": 200 }`). Descriptions:

- spark: "A small rocket built for small satellites. Cheap and flexible, but it cannot go beyond low orbits."
- rideshare: "Book a seat on a medium rocket together with other satellites. Very cheap per kilogram, but only to sun-synchronous orbit and only when the shared rocket flies (every 4 months)."
- arrow: "A light-medium launcher that can reach the Moon or Mars with small spacecraft. A good middle option when every dollar counts."
- workhorse: "A reliable, reusable medium-lift rocket. The default choice for most missions — affordable and flies often."
- titan: "A heavy-lift rocket for big observatories and deep-space missions. Powerful and expensive."

`components.json` — one object `{ buses, solarArrays, rtg, batteries, computers, adcs, propulsion, radios, hga, groundNetworks }` with the A8.4 values. Descriptions:

- bus_cube: "A shoebox-size satellite body. Tiny and cheap, fits one small instrument."
- bus_small: "A washing-machine-size body for focused missions with up to two instruments."
- bus_medium: "A car-size body with room for four instruments. Carries a spare reaction wheel."
- bus_large: "A bus-size body for flagship missions with six instruments. Carries a spare reaction wheel."
- sa_si: "Proven silicon cells. Cheap, but heavier and they lose 3% of their power every year."
- sa_gaas: "High-efficiency multi-layer cells: more than twice the power per kilogram, slower aging, double the price."

- rtg: "Makes electricity from the heat of decaying plutonium. Works in darkness and dust, far from the Sun. Very expensive."
- bat_liion: "Standard rechargeable battery. Use at most 40% of its charge each eclipse to make it last for years."
- bat_liion_adv: "Lighter high-energy cells that tolerate deeper discharge (50%)."
- obc_basic: "A reliable flight computer with 64 GB of storage."
- obc_adv: "Radiation-hardened, handles emergencies on its own, compresses data to half its size, 512 GB of storage."
- adcs_mag: "Electromagnets push against Earth's magnetic field. Light and cheap, but only roughly accurate (±5°) and useless far from Earth."
- adcs_wheels: "Spinning wheels turn the spacecraft; a star camera tells it where it points. Accurate to ±0.05°."
- adcs_precision: "Fine-guidance sensors and ultra-smooth wheels for telescopes: ±0.001°."
- prop_none: "No engine. Light and simple — but you cannot change your orbit, avoid debris or deorbit."
- prop_coldgas: "Puffs of compressed gas. Simple and safe but weak (Isp 65 s); holds at most 20 kg of gas."
- prop_mono: "Hydrazine thrusters (Isp 220 s). The classic choice for keeping a satellite in its orbit."
- prop_biprop: "Fuel and oxidizer burn together (Isp 315 s). Strong enough for big orbit changes like reaching GEO or Mars orbit."
- prop_electric: "Ion-like thrusters use electricity to push gas out very fast (Isp 1600 s): tiny propellant mass, but weak thrust — reaching orbit takes months and needs 1,000 W of power."
- radio_uhf: "A simple amateur-style radio. Very slow (9.6 kbps) and only works close to Earth."
- radio_sband: "A dependable radio for commands and modest data rates."
- radio_xband: "The standard science downlink: fast, works from Earth orbit to Mars."
- radio_kaband: "Very high data rates, but heavy rain at the ground station can interrupt the signal."
- hga: "A 1.5 m dish focuses the radio beam 20× more strongly. It must be aimed precisely (needs reaction wheels)."
- campus / commercial / deepspace: use the A8.2 descriptions.

`instruments.json` — 14 objects from A8.5; `tags` as arrays; add `whatItMeasures` and `description`:

id	whatItMeasures	description (shown on the card)
cam_wide	Takes wide colour pictures of large areas: clouds, coastlines, big fields.	Cheap, light and data-hungry. Good for context images.
cam_hires	Sees details down to about one metre: buildings, roads, single fields.	Needs very steady pointing and a fast downlink (200 Gbit/day).
imager_ms	Sees several colours, including invisible near-infrared, which reveals plant health and water.	The workhorse of farming and water monitoring.
imager_tir	Measures heat: soil dryness, cloud-top temperature, freezing dark craters.	Works day and night. Needs a cooler, so it uses 50 W.
radar_sar	Sends radio pulses and builds images from the echoes; works at night and through clouds and dust.	Powerful but heavy (150 kg) and power-hungry (400 W).
sounder_atm	Measures temperature and humidity at different heights: the main input for weather forecasts.	Best from an orbit that watches continuously.
lightning_map	Detects lightning flashes day and night, a sign that storms are strengthening.	Most useful staring at one region from far away.
spec_neutron	Counts neutrons escaping from the ground. A dip means hydrogen: a strong hint of water ice.	Light, low power, tiny data; does not need precise pointing.
lidar_alt	Fires laser pulses and times the echoes to map surface height very precisely.	Needs ±0.05° pointing — use reaction wheels.
magnetometer	Measures magnetic fields around the planet and in space.	Almost free in mass, power and data. Small science value.
spec_atm	Splits light into colours to identify gases like methane, carbon dioxide and water vapour.	Key for atmosphere studies on Earth and Mars.
det_particle	Counts energetic particles from the Sun; gives early warning of radiation storms.	Halves the damage risk from solar storms.
relay_uhf	Collects data from rovers and landers and forwards it to Earth.	Counts as an instrument; essential for Red Relay.

id	whatItMeasures	description (shown on the card)
tel_ir	A super-cold infrared telescope that sees heat from the most distant galaxies and from planets around other stars.	Huge (400 kg), $250M, needs ±0.001° pointing and a cold, stable orbit.

`scenarios.json` — six objects from A8.6. Field `briefing` (2–3 sentences, shown on the briefing screen) and `learningGoals` (3 bullet strings):

- **nile_watch** — briefing: "The Nile Delta feeds tens of millions of people, and every drop of water counts. Design a satellite that watches crop health and water use, so farmers know where and when to irrigate. You have $120M, a 2-year minimum mission, and a launch slot in 3 years." Goals: "Match the orbit to the observation (same lighting every day)"; "Balance instrument data with your radio link"; "Size power for eclipses".
- **storm_watch** — "Dust storms and flash floods can strike Africa and the Middle East within hours. Forecasters need an eye in the sky that never looks away. Watch continuously for at least 5 years." Goals: "Why geostationary orbit stares at one region"; "Delta-v: the spacecraft must finish its own orbit raising"; "Long missions need margins for aging".
- **ice_hunter** — "Future lunar crews need water for drinking, breathing and rocket fuel. It may be frozen in craters near the Moon's south pole that never see sunlight. Map it to help choose safe landing sites." Goals: "Deep-space communications"; "How instruments detect ice indirectly"; "Trading rocket cost against launch margin".
- **red_relay** — "Two jobs at once: study methane and dust in the Martian atmosphere, and relay data home for the rovers on the surface. Mars is unforgiving: weak sunlight, minutes of radio delay, and launch windows only every 26 months." Goals: "Solar power falls with distance from the Sun"; "Light delay demands onboard autonomy"; "Launch windows and schedule risk".
- **first_light** — "Light from the first galaxies has travelled for over 13 billion years and arrives as faint infrared heat. Fly a telescope cold and steady enough to catch it." Goals: "Why some telescopes go to L2"; "Pointing precision"; "Big science, big budget: protecting the reserve".
- **sandbox** — "No grades, no limits. Build anything and watch what happens." Goals: "Explore extremes"; "Find several designs that work"; "Break things on purpose and read the report".

`tagline` = the one-liners in A8.6 "Story". `userSegment` and `twins` = copy the A8.6 "User segment" line and the Mission Twins table exactly.

`difficulties.json` — three objects from A8.7. Descriptions: cadet "Basic parts only, extra budget, fewer surprises, hints, and rockets never fail."; engineer "All parts, normal budget, real launch risk."; director "Tight budget, more surprises, short decision timers. For experts."

`events.json` — ten objects. Use A8.8 for structure; text and full fields:

```
{
  "id": "solar_storm", "title": "Solar Storm!", "ownerRole": "engineer", "conceptId": "radiation",
  "description": "The Sun just released a burst of energetic particles. They will reach your spacecraft within
      hours and can damage sensitive detectors.",
  "options": [
    { "id": "safe_mode", "label": "Enter safe mode", "good": true,
      "description": "Switch the instruments off and point the arrays at the Sun until the storm passes (3 days
      without science).",
      "effects": [ { "type": "loseScienceDays", "days": 3 } ],
      "requires": { "autonomyIfDelayOverSec": 60 },
      "explanation": "Safe mode protected the instruments. Three days of lost data is a small price compared with
      a damaged detector." },
    { "id": "keep_observing", "label": "Keep observing", "good": false,
      "description": "Keep collecting data and trust the shielding.",
      "effects": [],
      "risk": { "chance": 0.25, "effects": [ { "type": "scienceMult", "value": 0.7 } ],
                "reductions": [ { "ifInstrument": "det_particle", "mult": 0.5 }, { "ifComputer": "obc_adv",
      "mult": 0.5 } ] },
      "explanation": "Lucky! The storm passed without damage and you kept collecting data.",
      "badExplanation": "A particle strike damaged a detector. Science output is permanently reduced by 30%." }
  ],
  "defaultOptionId": "safe_mode"
}
```

The other nine (write in the same structure):

- **micrometeoroid** (engineer, power_budget). Description: "A dust grain smaller than a grain of sand hit a solar panel at 10 km/s. The panel now gives 10% less power." autoEffects: powerMult 0.9. Options:

reduce_load ✓ "Reduce instrument use" — "Run instruments 15% less so the battery always stays charged." effects scienceMult 0.85 — explanation "Cutting the load kept the power budget positive. Engineers keep margins for exactly this." · **full_ops** "Keep full operations" — "Keep everything on and accept deeper battery drain." risk chance 0.05, chanceRule power_margin_after_hit, chanceIfRule 0.6, effects loseScienceDays 10 — explanation "Your power margin absorbed the hit; full operations continued." badExplanation "Without enough power margin the battery ran flat in the shadow. The spacecraft rebooted and lost 10 days of science." Default reduce_load.

- **ground_outage** (comms, ground_station). weightIfRadio radio_kaband 2. "A storm over the ground station (or urgent maintenance) means no contact for 3 days. Data will pile up in onboard memory." Options: **rent_backup** ✓ "Rent backup antenna time" — "$1.5M to use a partner agency's antenna." costM 1.5 — "Renting backup time kept the data flowing. Real missions sign backup agreements for this reason." · **wait** "Wait it out" — "Store data onboard for 3 days." contactLossDays 3 — "Memory held the data until contact returned (if your memory was big enough)." Default wait.
- **wheel_failure** (engineer, pointing). conditions adcsIn [adcs_wheels, adcs_precision]. "One of the reaction wheels that turn the spacecraft has stopped spinning." Options: **spare_wheel** ✓ "Switch to the spare wheel" — "Larger buses carry a backup wheel." requires busSpareWheel — "The spare took over. Carrying a backup (redundancy) saved the mission's pointing." · **magnetorquer_mode** "Get help from magnetorquers" — "Use Earth's magnetic field to assist (near Earth only). Pointing 5× worse." requires nearEarth, pointingMult 5 — "The magnetorquers hold the spacecraft steady, but less precisely." · **accept** "Accept degraded pointing" — "Pointing 20× worse; precise instruments will blur." pointingMult 20 — "The spacecraft wobbles more now, so sharp instruments lose quality." Default accept.
- **budget_cut** (planner, mission_cost). "Government funding was reduced. Finance asks your mission to give back 10% of its budget." Options: **pay_reserve** ✓ "Pay from our reserve" — requires reservePctOfBudget 10, effects costM value 10 reason "pct_of_budget" — "Your reserve absorbed the cut. That is exactly why planners keep one." · **shorten** "End the mission one year early" — lifetimeYearsDelta −1 — "Ending early saves operations money but costs a year of science." · **cheaper_ground** "Switch to a cheaper ground network" — downgradeGround — "Cheaper ground stations save money but give fewer contact hours, so less data comes home." Default shorten.
- **computer_reset** (comms, radiation). autoHandledByComputer obc_adv; autoHandledText "A cosmic ray flipped a bit in memory, but the rad-hard computer fixed it automatically. No science lost." Description "A cosmic ray flipped a bit in the computer's memory and it rebooted." Options: **reload** "Reload software from the ground" — loseScienceDays 1 + contactLossDays 1 — "The team reloaded the software; one day of science was lost." · **patch** ✓ "Upload a protective patch" — costM 0.5 + loseScienceDays 2 + immunity computer_reset — "The patch adds memory error checking, so this will not happen again." Default reload.
- **debris_conjunction** (planner, space_debris). conditions orbits [leo, sso, geo]. "Tracking shows a piece of old rocket debris may pass within 200 metres of your spacecraft in 12 hours." Options: **avoid** ✓ "Perform an avoidance burn" — requires propulsion; deltaVUse 2 + loseScienceDays 1 — "A 2 m/s burn moved you safely aside. Operators perform thousands of these every year." · **stay** "Stay on course" — risk chance 0.02 effects endMission reason "collision with space debris" — explanation "The debris passed by. The odds were low, but not zero." badExplanation "The debris hit the spacecraft. The mission is over, and the collision created even more debris." Default avoid; defaultOptionIdIfNoPropulsion stay.
- **discovery** (science, instrument_science). "Your instruments spotted something unexpected: an unusual signal worth a closer look." Options: **follow_up** ✓ "Follow up and send extra data" — scienceBonusPct 15 + dataMult 3 for 5 days — "The follow-up produced a bonus result. Flexible plans let missions chase surprises (if your downlink can handle the extra data)." · **stick_to_plan** "Stick to the schedule" — no effects — "Routine continued. Safe, but the surprise stays a mystery." Default stick_to_plan.
- **dust_storm** (science, instrument_science). conditions orbits [mars]. "A planet-wide dust storm is spreading across Mars. Rovers lose sunlight and go quiet, but the atmosphere is doing something scientists rarely see." Options: **observe** ✓ "Observe the storm" — scienceBonusPct 25 + relayMult 0.5 for 30 days — "Your orbiter recorded the storm from above: a rare dataset. The rovers were sending less anyway." · **relay_focus** "Focus on relay duty" — scienceBonusPct 5 — "You kept the relay channel ready, but most rovers were asleep to save power." Default relay_focus.
- **battery_aging** (engineer, battery). conditions needsEclipse true. autoEffects batteryMult 0.8. "After thousands of charge cycles the battery holds 20% less energy." Options: **eclipse_saving** ✓ "Switch off some instruments in eclipse" — scienceMult 0.9 — "Saving power in the shadow protects the aging battery." · **push_on** "Keep everything on" — risk chance 0.02, chanceRule battery_below_required, chanceIfRule 0.3, effects loseScienceDays 15 — explanation "The battery still had enough capacity; operations continued."

badExplanation "The battery ran out in eclipse and the spacecraft spent 15 days in emergency safe mode." Default eclipse_saving.

`checks.json` — object keyed by check id; each value `{ "title", "pass", "warn"?, "fail"?, "fixHint" }`. Placeholders are the A10 template variables.

id	title	pass	warn	fail	fixHint
launch_destination	Rocket can reach the orbit	The {vehicle} can deliver to {destination}.	–	The {vehicle} cannot fly to {orbit}. Each rocket can only reach certain destinations.	Pick a rocket that lists a capacity for {destination}.
launch_capacity	Rocket can lift the spacecraft	{totalKg} kg on a rocket that lifts {capKg} kg to this orbit ({launchMargin}% margin).	Only {launchMargin}% margin ({totalKg} of {capKg} kg). Spacecraft usually get heavier during development.	Too heavy: {totalKg} kg, but the {vehicle} lifts only {capKg} kg to this orbit.	Choose a bigger rocket, a lighter bus, fewer instruments, or a more efficient engine (less propellant).
bus_mass	Bus can carry the load	{totalKg} kg is within the {busName}'s {busMaxKg} kg limit.	{totalKg} kg is close to the {busName}'s {busMaxKg} kg limit.	The {busName} is rated for {busMaxKg} kg but the spacecraft weighs {totalKg} kg.	Pick a bigger bus or remove mass.
bus_slots	Room for instruments	{instrCount} of {slots} instrument slots used.	–	{instrCount} instruments, but the {busName} has only {slots} slots.	Remove an instrument or choose a bigger bus.
power_balance	Enough power	Arrays give {genW} W on average; the spacecraft uses {loadW} W ({powerMargin}% margin).	Only {powerMargin}% power margin. Solar arrays lose power every year in space.	The spacecraft needs {loadW} W but the arrays average only {genW} W.	Add solar array watts, switch to GaAs panels, or remove power-hungry instruments.
battery_eclipse	Battery lasts through the shadow	{batteryWh} Wh covers the {eclipseMin}-minute eclipse (needs {batteryReqWh} Wh).	The {eclipseMin}-minute eclipse needs {batteryReqWh} Wh; you have {batteryWh} Wh. Instruments will switch off in the shadow.	The battery ({batteryWh} Wh) is far too small for the {eclipseMin}-minute eclipse ({batteryReqWh} Wh needed).	Raise battery capacity to at least {batteryReqWh} Wh.
deltav_capability	Enough delta-v	The engine provides {dvCapability} m/s for the {dvRequired} m/s this mission needs.	Only {dvCapability} of {dvRequired} m/s: the orbit is reachable but the mission will end early.	Reaching {orbit} takes {insertionDv} m/s, but the spacecraft can only provide {dvCapability} m/s.	Choose stronger propulsion (higher Isp needs less propellant) or raise the delta-v margin.
electric_power	Electric thrusters have power	The arrays can drive the electric thrusters.	–	Electric thrusters need at least 1,000 W from the arrays at this distance from the Sun.	Increase the solar array or choose chemical propulsion.
adcs_environment	Attitude control works here	{adcsName} works at {orbit}.	–	Magnetorquers push against Earth's magnetic field, which is far too weak at {orbit}.	Choose reaction wheels.
pointing_quality	Instruments pointed precisely	{adcsName} (±{adcsDeg}°) meets every instrument's needs.	Blurred: {blurredList}. Your attitude control holds only ±{adcsDeg}°.	–	Choose a more precise attitude control system.
hga_pointing	Dish antenna can be aimed	The dish antenna can be aimed precisely.	The dish needs ±0.1° pointing; {adcsName} only gives ±{adcsDeg}°, so the dish does nothing.	–	Choose reaction wheels or better.
radio_range	Radio can reach Earth	{radio} via {ground}: {rateMbps} Mbps.	–	The {radio} through the {ground} cannot reach a spacecraft at {orbit}.	Beyond Earth orbit use the Deep-Space Network and an S, X or Ka-band radio.

id	title	pass	warn	fail	fixHint
data_return	Data gets home	{downlink} Gbit/day of downlink for {generated} Gbit/day of data ({returnPct}% returned).	Only {returnPct}% of the data can be sent home ({downlink} of {generated} Gbit/day).	Almost nothing gets home: {returnPct}% ({downlink} of {generated} Gbit/day).	Faster radio, a dish antenna, a bigger ground network, or the compression computer.
storage	Memory can buffer data	{storage} Gbit of memory is enough.	Data piles up faster than it can be sent; {storage} Gbit of memory fills within days.	–	Improve the downlink or choose the computer with more storage.
budget_total	Within budget	Total {totalCost} of {budget} ({reservePct}% reserve).	Only {reservePct}% reserve. Surprises during the mission may push you over.	Total cost {totalCost} is over the {budget} budget.	Cheaper rocket, fewer instruments, shorter mission or cheaper ground network.
role_allocation	{role} within allocation	{role} spends {roleCost} of {roleAlloc}.	{role} spends {roleCost}, over its {roleAlloc} allocation.	–	Request funds from another role, or cut costs in this role.
launch_window	Ready for the launch window	Ready at month {launchMonth}; the window is month {window}.	Development takes until month {launchMonth}, but the window is month {window}: {delayMonths} months of delay costing {delayCost}.	–	Fewer advanced parts and simpler instruments shorten development.
required_instruments	Mission objective covered	Every required instrument type is on board.	–	Missing a required instrument: {missingTags}.	Science Lead: add an instrument with that capability.
orbit_fit	Orbit suits the mission	{orbit} is ideal for this mission.	{orbit} works at {fit}% effectiveness; {recommendedOrbit} is better.	{orbit} cannot achieve this objective.	Commander: choose {recommendedOrbit}.
lifetime_min	Mission long enough	{lifetime} years planned (minimum {minLifetime}).	{lifetime} years is shorter than the required {minLifetime} years.	–	Extend the mission lifetime.
debris_disposal	Clean end of life	{disposalDv} m/s is kept to dispose of the spacecraft.	No way to deorbit at end of life. Dead spacecraft in crowded orbits become dangerous debris.	–	Carry propulsion with enough delta-v for disposal.
natural_lifetime	Orbit lasts long enough	The orbit is stable for the planned mission.	Without propulsion this orbit lasts only about {naturalLife} years, less than the planned {lifetime}.	–	Add propulsion or shorten the mission.

`glossary.json` — array of `{ id, term, category, short, long, example, related }`. Write these 33 entries (short = one line; long = 2–3 sentences; example = 1 sentence):

id	term	category	short	long	example
orbit	Orbit	orbits	The path a spacecraft follows around a planet.	A spacecraft in orbit is always falling, but moving sideways so fast that it keeps missing the ground. Higher orbits are slower and take longer per lap.	At 500 km one lap takes 95 minutes; at 35,786 km it takes a day.
leo	Low Earth Orbit	orbits	Orbits a few hundred km up.	Close to Earth: cheap to reach and great for detail, but thin air slowly drags spacecraft down.	The International Space Station flies at about 400 km.

id	term	category	short	long	example
sso	Sun-synchronous orbit	orbits	A polar orbit that crosses each place at the same local time.	Its tilt makes the orbit turn once a year with the Sun, so lighting is the same on every pass: ideal for comparing images over months.	Many Earth-imaging satellites pass overhead around 10:30 every morning.
geo	Geostationary orbit	orbits	One lap per day: hovers over one spot.	At 35,786 km above the equator a satellite moves with Earth's rotation, so it watches the same region nonstop.	Weather satellites watching Africa sit in GEO.
lagrange_point	Lagrange point L2	orbits	A balance point 1.5 million km beyond Earth.	At L2 the pull of the Sun and Earth lets a spacecraft keep pace with Earth. The Sun, Earth and Moon stay behind it, keeping telescopes cold and steady.	Large infrared space telescopes orbit around L2.
lunar_orbit	Lunar orbit	orbits	Orbiting the Moon.	Polar lunar orbits see the whole surface. The Moon's uneven gravity tugs on low orbits, so they need regular corrections.	Lunar orbiters have mapped ice-rich craters near the south pole.
delta_v	Delta-v	rockets	The "speed change budget" of a spacecraft, in m/s.	Every maneuver costs delta-v: entering an orbit, keeping it, avoiding debris, deorbiting. More delta-v needs more propellant.	Circularizing a geostationary orbit costs about 1,500 m/s.
rocket_equation	Rocket equation	rockets	Links delta-v, engine efficiency and propellant mass.	Propellant needed grows exponentially with delta-v: doubling delta-v needs much more than double the propellant. That is why big maneuvers make spacecraft heavy.	Our game uses it: propellant = dry mass × (e^(Δv / (Isp × 9.81)) − 1).
specific_impulse	Specific impulse (Isp)	rockets	How efficiently an engine uses propellant, in seconds.	Higher Isp means more delta-v from the same propellant. Electric thrusters have very high Isp but tiny thrust.	Cold gas: 65 s. Hydrazine: 220 s. Electric: 1,600 s.
payload_capacity	Payload capacity	rockets	How much mass a rocket can deliver to a destination.	The farther or higher the destination, the less a rocket can carry there. Always keep a margin.	A rocket that lifts 17,500 kg to low orbit may send only 2,500 kg toward Mars.
launch_window	Launch window	mission	The period when a launch can reach its target efficiently.	Planets move, so the cheapest path opens only at certain times. Mars windows come every 26 months.	Missing a Mars window means waiting more than two years.
reliability	Reliability	rockets	The chance something works as planned.	No rocket is perfect. A 98% reliable rocket still fails about 2 times in 100 launches. Reliability costs money.	Engineers add backups (redundancy) to improve reliability.
mass_budget	Mass budget	spacecraft	Adding up the mass of every part.	Every gram must be launched. Engineers keep a margin because spacecraft always grow heavier during development.	A 10% mass margin is a common minimum.

id	term	category	short	long	example
power_budget	Power budget	power	Comparing power generated with power used.	If parts use more power than the arrays produce on average, instruments must switch off. Keep a margin for aging and damage.	Our game warns below 15% power margin.
solar_power	Solar power	power	Electricity from sunlight.	Solar arrays lose a little power every year, and sunlight gets weaker farther from the Sun.	At Mars, sunlight is only 43% as strong as at Earth.
eclipse	Eclipse	power	Time spent in a planet's shadow.	In shadow, solar arrays produce nothing, so a battery must carry the load. Low orbits enter shadow on every lap.	In low Earth orbit an eclipse lasts about 36 minutes.
battery	Battery and depth of discharge	power	How much of the battery you use each cycle.	Using only part of the charge each eclipse lets batteries survive thousands of cycles. Our Li-ion battery uses 40%.	A 350 Wh battery at 40% safely gives 140 Wh per eclipse.
rtg	Radioisotope power	power	Electricity from the heat of decaying plutonium.	Works in darkness and dust, far from the Sun, for decades. Very expensive and heavy.	Rovers and outer-planet probes have used them.
data_rate	Data rate and data volume	data	How fast data can be sent (Mbps) and how much is made per day (Gbit).	If instruments make more data than the radio can send during contact time, data piles up and is eventually lost. Compression helps.	150 Mbps for 24 minutes a day sends about 216 Gbit.
ground_station	Ground station	data	An antenna on Earth that talks to spacecraft.	A spacecraft can only send data while a station can see it. Bigger dishes hear fainter signals from farther away.	Deep-space networks use 34–70 m dishes on three continents.
antenna_gain	Antenna gain	data	How strongly an antenna focuses its beam.	A dish concentrates power into a narrow beam, which multiplies the data rate but must be aimed precisely.	Our 1.5 m dish multiplies the rate by 20.
light_delay	Light delay	data	Radio signals travel at the speed of light, which takes time.	Far from Earth, commands arrive minutes later, so the spacecraft must react to emergencies on its own.	Mars is 3 to 22 light-minutes away.
pointing	Pointing accuracy	spacecraft	How precisely the spacecraft can aim.	Cameras, lasers, telescopes and dish antennas need steady pointing. Attitude control systems differ by 5,000×.	A telescope may need ±0.001°: like aiming at a coin 1 km away.
radiation	Space radiation	spacecraft	Energetic particles from the Sun and deep space.	Particles can flip bits in computers or damage detectors. Far from Earth's magnetic field, radiation is stronger.	Radiation-hardened computers resist bit flips.
space_debris	Space debris	mission	Dead satellites and fragments still in orbit.	Debris moves at about 10 km/s, so even small pieces are dangerous. Responsible missions deorbit at end of life and dodge close approaches.	Many agencies now require deorbiting within a few years of mission end.

id	term	category	short	long	example
mission_cost	Mission cost	mission	Hardware, integration, launch and years of operations.	Integration and testing add about 20% to hardware cost. Operations and ground stations cost money every year. A reserve protects against surprises.	Our game warns when the reserve drops below 5%.
mission_lifetime	Mission lifetime	mission	How long the mission operates.	Longer missions return more science but cost more to operate, need more propellant, and face more aging and surprises.	Our scenarios set a minimum lifetime.
mission_requirements	Mission requirements	mission	What the mission must achieve to succeed.	Requirements come from the science goal: which measurements, where, for how long, for how much. Every design choice must satisfy them.	Ice Hunter requires a neutron spectrometer.
instrument_science	Instruments and science	spacecraft	What each instrument can measure.	Different instruments answer different questions. The right one for the goal beats a bigger, wrong one.	Thermal infrared shows heat; multispectral shows plant health.
concept_maturity	Concept Maturity Level (CML)	mission	How far a mission idea has been worked out, from napkin sketch to point design.	NASA's JPL rates concepts from CML 1 "Cocktail Napkin" through CML 2 "Initial Feasibility" and CML 3 "Trade Space" to CML 4 "Point Design". A level counts as reached when about 80% of its checklist is green.	Our readiness review tells you which CML your design has reached.
trade_space	Trade space	mission	All the different designs that could meet the goal, and how they compare.	Exploring the trade space means changing one thing at a time and watching how science, cost and risk respond, before picking one design. Skipping it gives weaker, costlier missions.	The Trade-Space Explorer shows science vs battery size, rocket, radio and more.
mission_segments	Mission segments	mission	Launch, space, ground and user: the four parts of every mission.	The launch segment gets you to space, the space segment collects data, the ground segment talks to the spacecraft, and the user segment is the people who use the data. Design all four together.	In our game each role owns a segment; the scenario names its users.
trade_off	Trade-off	mission	Gaining one thing by giving up another.	Every choice in mission design improves something and costs something else. There is rarely a single perfect design.	A better camera means more mass, power, data and cost.

`dialog.json` — object of arrays of `{ speaker, expr, text, target? }`:

`tutorial` (speaker `hassan`; targets as listed):

1. (no target, happy) "Welcome to mission control! I'm Dr. Hassan Nasr, your flight director. Today we design a satellite to watch over the Nile Delta's farms and water."
2. (`meter-mass`, neutral) "These seven meters are your limits. Every choice moves several of them at once — that's real mission design."
3. (`tab-planner`, neutral) "Your Mission Commander picks the orbit, the mission length and the rocket. A sun-synchronous orbit gives the same lighting every day — perfect for comparing crops."

4. (`vehicle-cards`, neutral) "Each rocket lifts a certain mass to each orbit. Heavier spacecraft need bigger, pricier rockets."
5. (`tab-science`, neutral) "The Science Lead chooses instruments. Each one adds science — and also mass, power, cost and data."
6. (`instrument-cards`, happy) "Hover over the Thermal Infrared Imager. See the meters preview the change before you even click?"
7. (`tab-engineer`, neutral) "The Spacecraft Engineer feeds the instruments: solar arrays for power, a battery for the shadow, propellant for keeping the orbit."
8. (`meter-power`, worried) "If the load is higher than generation, instruments switch off. Keep at least 15% margin — arrays weaken every year."
9. (`tab-comms`, neutral) "Data is worthless until it reaches Earth. The Comms Officer picks the radio and the ground stations."
10. (`meter-data`, worried) "Our starter S-band radio cannot keep up with the imager. Try the X-band transmitter and watch this turn green."
11. (`review-button`, neutral) "When you're ready, open the readiness review. Red means NO-GO — read why, fix it, then sign off."
12. (`launch-button`, happy) "Then launch! Surprises will happen during the mission. Choose wisely — and remember, there's no single perfect design."

`intro_<scenarioId>` (2 lines each, speaker hassan):

- nile_watch: "The Nile Delta feeds millions, and water is precious. A satellite that sees crop health can tell farmers where to irrigate." / "We need a multispectral imager or a camera, at least two years in orbit. The budget is modest — spend wisely."
- storm_watch: "Dust storms and flash floods can strike in hours. Forecasters need eyes that never blink." / "Carry a sounder or a thermal imager and watch for five years. Which orbit lets you stare at one region all the time?"
- ice_hunter: "Future crews will need water on the Moon — to drink, to breathe, even to make rocket fuel." / "Bring a neutron spectrometer to sniff out hydrogen, plus an altimeter or thermal imager to map the craters. Deep-space comms are a must."
- red_relay: "Two jobs at once: study the Martian air, and relay data for the rovers below." / "Mars is far: weak sunlight, a 12-minute radio delay, and launch windows every 26 months. Miss it and we wait two years."
- first_light: "We want light that left the first galaxies 13 billion years ago. It arrives as faint infrared heat." / "An infrared telescope must be very cold and very steady. Somewhere far from Earth's warmth would be ideal…"
- sandbox: "Sandbox mode: no limits. Build anything and see what happens." / "Try extremes. No battery? A giant radar on a tiny bus? The report will tell you why."

Other keys: `council_solo` (hassan, neutral): "The agency gives you {budget}. Here is a suggested split between your four roles — adjust it if you like." · `launch_nogo` (hassan, worried): "We still have NO-GO items. Launching now will probably end badly. Are you sure?" · `launch_go` (hassan, happy): "All stations GO. Good luck, crew!" · `report_success` (hassan, happy): "Mission accomplished! Read the report — even a success has lessons." · `report_partial` (hassan, neutral): "We got some science home, but not everything. Let's see what to change." · `report_failure` (hassan, worried): "Every space agency has lost missions. What matters is understanding why — let's read the report." · `review_cml4` (hassan, happy): "Point design reached: CML 4. This concept is mature enough to fly." · `review_cml3` (hassan, neutral): "We're still exploring the trade space. Turn the yellows green before we commit." · `review_cml2` (hassan, worried): "This is still an early feasibility sketch. Let's fix the red items first." · `twin_intro` (hassan, happy): "Your design is a close cousin of a real mission:" · `priya_tip` (speaker priya, neutral) shown on the comms tab when data_return is warn/fail: "Ground station here! We can only listen while your spacecraft is in view. A faster radio or a bigger network gets more of your data home."

`characters.json` — 11 objects `{ id, name, from, role, playable, title, bio, lines: { success, failure, warning } }`:

id	title	bio	success	failure	warning
nour	Mission Commander	Former air-traffic controller from Alexandria who loves a plan with a backup plan.	Right on budget and on time. That's how it's done.	We'll plan smarter next time.	Our reserve is getting thin.

id	title	bio	success	failure	warning
lucas	Mission Commander	Ex-test pilot from Toulouse; calm under pressure, allergic to wasted money.	Textbook mission.	Debrief time. What did we miss?	That rocket is costing us a fortune.
diaa	Mission Commander	Seasoned leader from Cairo who turns bold ideas into missions that launch on time and on budget.	Bold idea, solid plan, perfect launch. Well done, team.	Every great mission comes back stronger. Back to the drawing board.	Let's check the numbers before we commit.
omar	Spacecraft Engineer	Mechanical engineer from Aswan who builds satellites like puzzles.	Every bolt held. Beautiful.	I'll find the weak part and fix it.	We're too close to the limit — I need margin!
elena	Spacecraft Engineer	Veteran propulsion engineer; has a rocket equation tattoo (on her coffee mug).	Margins saved us. They always do.	The numbers told us. We should have listened.	Not enough delta-v. Physics does not negotiate.
amara	Science Lead	Planetary scientist from Lagos who has mapped craters on two worlds.	Look at this data! Papers for years.	We'll get the science next time.	We're missing the instrument we need for this goal.
hiroshi	Science Lead	Astronomer from Sapporo who built telescopes before he could drive.	A clear view. Wonderful.	The sky will wait for us.	My instrument needs steadier pointing.
kwame	Comms & Data Officer	Radio engineer from Accra who can hear a whisper across the Solar System.	Every bit made it home.	Lost signal… we'll fix the link.	The radio can't keep up with the data.
layla	Comms & Data Officer	Network specialist from Abu Dhabi who schedules antennas like a chess game.	Downlink complete — 100%.	We need a better link.	Data is piling up in memory.
hassan	Flight Director	Thirty years in mission control in Cairo. Has seen every kind of failure — and learned from all of them.	Mission accomplished, crew.	Failure is a teacher. Let's read the lesson.	Check the readiness review before we go.
priya	Ground Station Operator	Runs the antennas that listen to our spacecraft; knows every pass by heart.	Signal strong and clear.	We lost contact.	Your spacecraft is only in view for minutes a day.

`from` values: Egypt, France, Egypt (Diaa), Egypt, Russia, Nigeria, Japan, Ghana, UAE, Egypt, India. `playable` false for hassan and priya; `role` "npc" for them.

Step 3 (H2–H9): sources, GDD, text review

`docs/SOURCES.md`**.** Validity is judged on "sound science, reliable data". List each source with what we used it for, and open every link to confirm it works before submitting. Start with:

- **Official challenge resources (cite first, with what we used from each, see A3):**
  - R. R. Wessen, C. Borden, J. Ziemer, J. Kwok, "Space Mission Concept Development Using Concept Maturity Levels", Jet Propulsion Laboratory, California Institute of Technology, 2013: CML 1–4, trade-space loop, checklist with the 80% rule.
  - J. Levri, "Design and Develop Science Missions", NASA Small Spacecraft Systems Virtual Institute, Access to Space for All Systems Engineering Webinar Series, 26 March 2025: mission segments, design process, requirements with verification.
  - NASA Webb, "Build It Yourself" game: https://science.nasa.gov/mission/webb/game-build-it-yourself/ (science drives design, infeasible options greyed out, real mission revealed after launch).
  - NASA, Web-based Space Mission Visualization: https://nasa.github.io/mission-viz/ (Keplerian orbit visualization and propagation, Earth–Moon transfer).
- NASA Space Apps 2026 challenge page: required.
- Mission Twin facts (Landsat 9, Sentinel-2, GOES-16, MTG-I1, Lunar Prospector, LRO, MRO, ExoMars TGO, MAVEN, JWST, Herschel): link each mission's official page and double-check every year and fact in A8.6 before submitting.
- NASA Small Spacecraft Technology State-of-the-Art report (NASA Ames): component classes, masses, power, data rates.
- NASA "Basics of Space Flight" (JPL): orbits, Lagrange points, light delay, deep-space communications.
- NASA Deep Space Network overview: dish sizes, three-site coverage.

- ESA Space Debris Office: debris risk and end-of-life disposal practice.
- Public launch-vehicle user guides (small, medium, heavy classes): capacities by destination (our vehicles are generic classes, not brands).
- Mission facts used as anchors: polar lunar orbiters mapping hydrogen, Mars orbiters relaying rover data and their downlink rates, L2 infrared observatories, geostationary weather imagers with lightning mappers, sun-synchronous Earth-observation satellites.
- The rocket equation (Tsiolkovsky), as used in A9.
- State clearly in README: "Values are simplified game values inspired by these public sources."

Text review: after the first build (H6), play every screen and fix wording in JSON only. Rules: plain English, short sentences, numbers with units, explain *why*, never blame the player.

Step 4 (H6–H13): balance and decisions

- At H13 (2 AM), read Programmer A's balance table. Targets: a thoughtful design scores 650–850 on its home difficulty; the default design scores 300–500; Cadet nile_watch can reach 3 stars; each scenario has at least two different designs above 650.
- Tune only data values (science targets, budgets, event chances), never formulas, and post each change in `#bible-changes` (version bump). Programmer A's A17 tests may need new expected values; tell them.
- **You make the call** at 1:00 AM (H12) on the online cutoff rule (A16), based on QA's multiplayer test.

Step 5 (H14–H21, 3–10 AM): README, summary, video, slides

`README.md` **template:**

```
# Delta-V Tycoon
Design, launch and fly a space mission, alone or with up to 3 friends online.
**Play:** https://<user>.github.io/delta-v-tycoon/  ·  **Video:** <link>  ·  **Challenge:** NASA Space Apps 2026:
      Space Mission Design Game
## What it is
<project summary below>
## How to play (2 minutes)
1. Pick a scenario and difficulty. 2. Split the budget. 3. Design: orbit, rocket, bus, power, instruments, comms.
4. Read the readiness review, fix NO-GO items, sign off. 5. Launch and handle surprises. 6. Read the report and
      try again.
## Team play
Create a room, share the 5-letter code, claim roles (Commander, Engineer, Science, Comms), agree on the budget,
      sign off together.
## The science inside
Rocket equation, power and eclipse budget, link budget and contact time, pointing accuracy, launch windows, space
      debris rules (see docs/SOURCES.md).
## Built with
React, TypeScript, Vite, Zustand, Firebase Realtime Database, Recharts, lz-string; fonts Press Start 2P and VT323
      (SIL OFL).
## Tools used
<list every tool the team used, including generative tools, as required by the event code of ethics>
## Team
<names and roles>
## Credits & licence
See CREDITS.md. Code: MIT.
```

Project summary (draft, about 170 words; adjust to the final build):

> *Space missions are systems of trade-offs: a better instrument adds science, but also mass, power, data and cost, which ripple into the rocket, the battery and the radio. Students rarely get to feel these connections.* ***Delta-V Tycoon*** *is a free browser game where players, alone or as an online crew of up to four, design and fly a complete mission. They choose one of five missions inspired by real ones, from watching Nile Delta farms to mapping lunar ice and relaying Mars rover data. They split a budget between four roles, build the spacecraft part by part, and pick an orbit and a rocket while seven live meters show mass, power, budget, data, comms, time and delta-v. A readiness review, built on NASA JPL's Concept Maturity Levels, explains every NO-GO in plain language and tells the team when its concept reaches a flyable point design. Then they launch, handle random events such as solar storms and debris, and receive a report explaining why the mission succeeded or failed and what to try next. After launch, the game reveals the real mission their design resembles. Real formulas (rocket equation, power and link budgets) make every decision matter, and there is never one perfect design.*

30-second video (1920×1080, English captions burned in, calm CC0 music, no people under 18, no agency logos). Record with OBS from the public build at 1920×1080, browser zoom 100%, a clean browser profile without bookmarks.

Time	Picture	Caption / voice
0–3 s	Logo on the title screen, crew sprites idle	"Can you build a space mission that survives?"
3–7 s	Lobby: 4 players join, claim roles (fast cuts)	"Up to 4 players. 4 roles. One budget."
7–14 s	Design: hover an instrument → meters preview; mass bar turns red; switch rocket → green	"Every choice changes mass, power, cost, data…"
14–18 s	Review panel: CML meter climbs to CML 4, NO-GO fixed, four sign-offs	"…and NASA's maturity checklist explains why."
18–22 s	Launch animation	(launch sound)
22–26 s	Ops: solar storm event, choose "Safe mode", charts moving	"Handle surprises in real time."
26–30 s	Report: stars, Why section, Mission Twin card; end card with URL + team name	"Delta-V Tycoon: play free at <short URL>."

Upload to YouTube as **Public** (or Unlisted, which still opens without sign-in), title "Delta-V Tycoon | NASA Space Apps Cairo 2026", description = summary + links.

7 slides (prepare them even if you submit the video: they are the backup demo and the People's Choice deck). Use the game's palette and screenshots; no agency logos.

1. **Delta-V Tycoon**: logo, one-liner, team names and roles, challenge name.
2. **The problem**: mission design is a web of trade-offs; students meet the pieces separately, never the system.
3. **Our solution**: the game loop diagram (Briefing → Council → Design → Review → Launch → Ops → Report → Retry).
4. **Built on NASA's own method**: Need & Goals → Concept → Requirements → Design → Build & Verify → Implement (Levri 2025); launch / space / ground / user segments; Concept Maturity Levels 1–4 with the 80% green rule (JPL). Plus real engineering: delta-v → propellant → rocket chain, power vs eclipse, data vs downlink, pointing, launch windows, debris.
5. **Play together**: online rooms, 4 roles with conflicting goals, Budget Council, funds requests, sign-offs, event ownership.
6. **What players learn**: screenshots of the CML meter, a NO-GO explanation, the Trade-Space Explorer and a report with its Mission Twin; five scenarios × three difficulties; Academy.
7. **Impact & next steps**: free and runs in any browser; classroom use; next: more scenarios from new missions, Arabic interface, teacher dashboard. Links (play, GitHub, video).

Step 6 (H22–H23): submission (you only)

1. Final build tagged `golden-final`; QA confirms it is the one online.
2. **Local Cairo form:** demo link (video or slides), final project link (game URL), GitHub repo, summary, resources, team info. Submit.
3. **Global NASA Space Apps site:** the same demo, links, summary, resources, tools-used disclosure. Submit.
4. QA opens every submitted link in a private window on another device and confirms it works without sign-in. Screenshot the confirmation pages.
5. Post "SUBMITTED" with screenshots in `#general`. Nothing changes after this.

People's Choice (if shortlisted, ~5 PM): 2-minute pitch: 20 s problem, 60 s live demo (pre-opened tabs: a design with one NO-GO ready to fix, then launch at 4×), 20 s team play, 20 s impact. Have the video ready as backup if the Wi-Fi fails, and a phone hotspot.

B5. Your daily rhythm during the event

- Every 2 hours: read every status ping; unblock people first; update a short progress list in `#general`.
- Protect sleep shifts (max 2 asleep at once) and the **3:00 AM freeze**.
- Say **no** to new features after H11 (midnight) unless they are on the P1 list and the owner is ahead.
- Keep a list of decisions you made and why (useful for the pitch).

PART A — SHARED PROJECT BIBLE (identical in all six briefs)

A0. What changed in v1.1 (read this first)

- **Official NASA resources integrated** (A3): JPL's *Concept Maturity Levels* paper, NASA's *Design and Develop Science Missions* webinar, NASA Webb's *Build It Yourself* game and NASA's *mission-viz* orbit visualizations. They now shape the game structure, not just the sources list.
- **New features:** F-33 Mission Twin (the real mission your design resembles), F-34 Trade-Space Explorer (how science changes when you change one parameter), F-35 Concept Maturity meter in the readiness review (CML 2 → 4), F-36 requirements with a "Verified by" column, NASA process labels on the phase stepper, a "Who uses your data" line (user segment), Kepler-ellipse orbit drawing and transfer animation.
- **New playable character: Diaa El Awdi**, a third Mission Commander (he agreed to appear). Character files: 33 portraits, 11 idle sprites, 9 suit sprites.
- **New schedule: feature freeze at 3:00 AM Saturday (H14).** Everything after 3 AM is bug fixes, footage, video, slides and submission. All deadline tables were updated.

A1. How to use this document

This brief has two parts. **Part B (before this)** is your personal role: what you build, in what order, and how you know it is done. **Part A (this part)** is the single source of truth that all six roles share: the game design, every data value, every formula, every file name and every code interface.

- Part A is **identical in all six briefs**. If two people read Part A, they build parts that fit together without talking.
- If Part A and Part B ever disagree, **Part A wins**. Report the conflict to the Lead.
- Only the Lead may change Part A during the event. Every change is posted in the team Discord channel `#bible-changes` with a version number (v1.0 → v1.1 ...). Code and assets follow the latest posted version.
- All IDs in this document (`imager_ms`, `res_power`, `nile_watch` ...) are **exact strings**. Copy them character for character. File names, JSON keys and TypeScript identifiers must match exactly or integration breaks.
- The brief is written to be complete and self-contained: it can be handed to any collaborator or tool and executed without extra context. When you hand it over, include **both** Part B and Part A.

A2. Event facts and hard rules

Item	Value
Event	NASA Space Apps Cairo 2026 (local event of the global NASA Space Apps Challenge)
Venue	Elsewedy University of Technology (SUT), Cairo
Onsite registration	Friday 13 Nov 2026 from 9:00 AM (guidebook says 8:00 AM — arrive at 8:00)
Hackathon starts	**Friday 13 Nov, 1:00 PM** (call this H0)
Mentorship opens	Friday 2:00 PM, via Discord (plus onsite at peak hours)
Submission deadline	**Saturday 14 Nov, 1:00 PM** (H24). Late = rejected, no exceptions
Feature freeze	**Saturday 3:00 AM** (H14) — after this, only bug fixes, video, slides, submission
Our internal deadline	**Saturday 12:00 PM** — submission done, links verified
Closing ceremony	Saturday ~5:00 PM (People's Choice shortlist presents live)
Challenge	**Space Mission Design Game** (beginner / intermediate)
Project name	**DELTA-V TYCOON**
Team size	6 — max **5 onsite**; the 6th member works virtually on day 1 (may join onsite day 2)

Hard rules (breaking any of these can disqualify the team):

1. **No production work before H0 (Friday 1:00 PM).** Planning, splitting tasks, creating accounts, installing tools and creating an *empty* repository are allowed. Writing game code, drawing final art or recording before

H0 is not. Submitting a ready-made project is forbidden.

2. **Stick to the registered idea and challenge.** The idea cannot be changed after registration.
3. **Demo:** exactly **7 slides OR a 30-second video**, uploaded to a publicly accessible platform. It must not contain images, names or voices of anyone under 18.
4. **Language:** the solution must be in English. Any Arabic voice in the video needs English subtitles.
5. **Every submitted link must open without a password, sign-up or access request** (GitHub repo public, game link public, Drive files "anyone with the link").
6. **Submit twice:** first the local Cairo form, then the same material on the global NASA Space Apps site (needed for global judging). **Only the team leader submits.** No changes are allowed after submission.
7. **Work honestly:** credit every source and asset, respect copyright, and **disclose generative tools used** (the event code of ethics requires it). We list all tools in `README.md` → "Tools used".
8. **Do not use the NASA logo** ("meatball") or any agency logo in the game, video or slides — it implies endorsement.
9. Mentors can guide but will not write code.

Judging criteria (NASA Space Apps): Impact · Creativity · Validity (sound science, reliable data, feasible) · Relevance (answers the chosen challenge) · Presentation. Every feature below exists to score on at least one of these.

A3. The challenge, and how we answer every part of it

The challenge asks for an interactive game in which participants design, manage and simulate a complete space mission, make engineering decisions, manage limited resources, and see how each choice shapes mission success.

Official NASA resources and how the game uses each one (cite all four in README, `docs/SOURCES.md`, slides and the submission; this is our Validity and Relevance evidence):

Resource	Key idea	Where it lives in our game
R. Wessen, C. Borden, J. Ziemer, J. Kwok (JPL), *Space Mission Concept Development Using Concept Maturity Levels* (2013)	Concepts mature in levels: CML 1 Cocktail Napkin (science question, unique idea, launch timeframe, cost class) → CML 2 Initial Feasibility (key parameters, first calculations) → CML 3 Trade Space (explore how science, cost and risk change) → CML 4 Point Design (one design with acceptable margins). CML 2–4 loop back and forth. "Science partials" = how science return changes with one parameter. A checklist with green/yellow/red; a level is reached at ~80% green. Independent assessment beats self-assessment.	Briefing = CML 1; live meters = CML 2; Trade-Space Explorer = CML 3 (F-34); readiness review = CML 4 checklist with green/yellow/red and the 80% rule (F-35); the review is an independent assessor (Dr. Hassan's review board), not self-scored
J. Levri (NASA Small Spacecraft Systems Virtual Institute), *Design and Develop Science Missions*, Systems Engineering Webinar, 26 Mar 2025	A science mission = launch segment + space segment + ground segment + user segment. Process: Identify Need & Goals → Define Concept → Define Requirements → Design → Build & Verify → Implement. "Begin with the end in mind": start from what data users need. Every requirement has a verification method.	Phase stepper uses these six process names (A6). The four game roles map to the segments (A7). Each scenario names its user segment ("Who uses your data"). Requirements panel shows "Verified by" (F-36)
NASA Webb, *Build It Yourself* game (science.nasa.gov/mission/webb/game-build-it-yourself/)	Science requirements drive telescope design; infeasible options are greyed out; after launch the game shows a real mission with similar data; levels grow from small to large missions	Locked / "cannot reach" options with a reason; Mission Twin card after launch and in the report (F-33); difficulty levels from small (Cadet) to flagship (Flight Director)
NASA *Web-based Space Mission Visualization* (nasa.github.io/mission-viz/)	Browser orbit visualizations from Keplerian elements with an orbit propagator; Earth-to-Moon transfer animation	Orbit view draws Kepler ellipses and moves the spacecraft with Kepler's equation (faster near the planet); transfer animation for far orbits (B's brief)

Traceability matrix (challenge item → our feature ID, see A5). Judges check Relevance against exactly this list.

Challenge asks for	Our feature
Define a mission objective, then requirements	F-01 scenario briefing, F-02 requirements panel
Design a spacecraft (structure, power, batteries, comms, computer, attitude control, instruments)	F-03 spacecraft builder, F-04 instruments
Instruments differ in mass, power, cost, data and science value	F-04 (all five values shown on every card)
Choose orbit; explain coverage, observation, comms, lifetime	F-05 orbit picker with pros/cons + orbit view
Launch vehicles with capacity, cost, orbit capability; check mass vs vehicle	F-06 + check `launch_capacity`, `launch_destination`
Manage budget, mass, power, comms, data, time (lifetime)	F-07 live resource dashboard (7 meters incl. delta-v)
Every decision has a consequence; no single perfect solution	Formulas A9 (each choice moves 3+ meters); reference designs A17 show several winning designs
Clear UI, configuration, resource management	F-03..F-08
Single player and team of up to 4, roles, negotiation, collective decisions	F-18 online rooms, F-19 roles, F-20 Budget Council, F-21 sign-offs, F-22 role goals, F-23 same-screen mode
Simulation and evaluation	F-10 launch, F-11 operations sim, F-13 scoring
Feedback that explains *why*	F-08 check explanations, F-14 report "Why" section
Modify and re-simulate many times	F-15 retry keeping design, compare attempts
Difficulty levels, simple vs advanced components	F-16 Cadet / Engineer / Flight Director, basic/advanced tiers
Multiple scenarios	F-17 five scenarios + sandbox
Random events / unexpected constraints	F-12 ten events with decisions
Spacecraft visualization, real-time dashboards	F-09 composited spacecraft + orbit view, F-11 charts
Leaderboards, mission reports, save/share/compare	F-25, F-14, F-24
Competitive scenario (teams, same objective)	F-25 challenge codes (same seed for every team)
Sandbox mode	F-30
Use official NASA resources (A3)	F-33 Mission Twin, F-34 Trade-Space Explorer, F-35 Concept Maturity meter, F-36 verified requirements, NASA process stepper
Accessible to all knowledge levels; concepts taught in-game	F-26 Mission Academy, tooltips, F-27 tutorial

A4. Game concept

One-liner: *Delta-V Tycoon* puts you and up to three friends in charge of a small space agency's mission control. Pick a real-world-style mission, build the spacecraft part by part, pick an orbit and a rocket, argue over a shared budget, sign off together, launch — and watch every decision play out in a simulated mission, with a report that explains exactly why it succeeded or failed.

Why "Delta-V": delta-v (change in velocity, in m/s) is the "fuel budget" of space travel. In our game every orbit costs delta-v, the rocket equation turns delta-v into propellant mass, and propellant mass pushes you toward a bigger, more expensive rocket. This chain is the heart of real mission design and is visible on screen.

Why "Tycoon": you manage limited money like a business. Budget is split between the four roles, you can request money from teammates, and leftover reserve improves your score.

Core loop (one mission, 10–20 minutes):

1. **Briefing** — choose scenario + difficulty; read the need, the goals, who uses the data, and the requirements (F-01, F-02). *(NASA: Identify Need & Goals · CML 1 Cocktail Napkin)*
2. **Crew** — solo: pick your avatar; team: create/join a room, claim roles (F-18, F-19).
3. **Budget Council** — split the budget between the 4 roles; everyone agrees (F-20).
4. **Design** — each role configures its part; all 7 resource meters update live; the Trade-Space Explorer shows how science changes with any one parameter (F-03..F-07, F-09, F-34). *(NASA: Define Requirements & Design · CML 2–3)*
5. **Readiness review** — automatic GO/NO-GO checks with explanations, the Concept Maturity meter, and sign-offs from every role (F-08, F-21, F-35). *(NASA: Build & Verify · CML 4 Point Design)*
6. **Launch** — animated launch; can fail on real reliability numbers (F-10).
7. **Operations** — the mission plays out day by day at high speed; random events demand decisions from the role that owns them (F-11, F-12).
8. **Report** — score, stars, outcome, what went well, what hurt and why, what to try next, the real Mission Twin your design resembles, and who used your data (F-13, F-14, F-33).

9. **Iterate** — retry with the same design, change something, compare scores (F-15).

Game modes:

Mode	Description	Priority
Solo mission	One player owns all 4 roles	P0
Online team room	1–4 players on their own devices, room code + link, roles, council, sign-offs	P0-online (see cutoff rule A16)
Same-screen team	Same team mechanics on one device (fallback if online fails)	P0 (it is the solo code path)
Challenge code	Same scenario + difficulty + seed for every team → fair competition	P1
Sandbox	No budget limit, all components unlocked, events optional	P1
Agency career ("Tycoon")	Play scenarios in order; reputation raises next budget	P2 (only if everything else is done)

A5. Feature list (IDs, priority, owner)

Owners: **L** = Lead, **A** = Programmer A (core + rooms), **B** = Programmer B (UI + build), **C** = Character artist, **E** = Environment/asset artist, **Q** = QA tester.

ID	Feature	Pri	Owners
F-01	Scenario select + briefing (Dr. Hassan presents the mission)	P0	B, L (text), E (cards), C
F-02	Requirements panel (required instrument types, orbit, lifetime, budget, window)	P0	B, A
F-03	Spacecraft builder: bus, solar array (type + watts), RTG, battery (type + Wh), computer, attitude control, propulsion + delta-v margin	P0	A, B, E
F-04	Instrument catalog (cards show mass, power, cost, data/day, science, pointing need)	P0	A, B, E
F-05	Orbit picker with pros/cons and live orbit preview	P0	A, B, E
F-06	Launch vehicle picker with capacity per destination	P0	A, B, E
F-07	Resource dashboard: Mass, Power, Budget, Data, Comms, Time, Delta-v	P0	A, B, E
F-08	Readiness review: checks with severity, explanation, fix hint, concept link	P0	A, B, L
F-09	Spacecraft preview (composited pixel sprite) + orbit view canvas	P0	B, E
F-10	Launch sequence (animation, success or failure)	P0	B, E, A
F-11	Operations simulation with charts and speed control	P0	A, B
F-12	Random events with options, owner role, timer, suggestions	P0	A, B, L, E
F-13	Score (0–1000), stars, outcome class	P0	A, B
F-14	Mission report: breakdown, timeline, Why, Try next, print to PDF	P0	A, B, L
F-15	Retry with same design; attempt history and comparison	P1	B
F-16	Difficulty levels and component tiers	P0	A, B, L
F-17	Scenarios: nile_watch, storm_watch, ice_hunter (P0); red_relay, first_light (P1); sandbox (P1)	P0/P1	L, A
F-18	Online rooms: create/join by code or link, presence, max 4	P0-online	A, B
F-19	Roles: claim/release, role tabs, owner badge	P0	A, B, C
F-20	Budget Council + funds requests between roles	P0-online	A, B
F-21	Sign-offs (reset on any design change); event ownership; suggestions	P0	A, B
F-22	Role goals (personal objectives) feeding the team score	P0	A, B, L
F-23	Same-screen team mode (LocalRoom)	P0	A, B
F-24	Save designs, share codes/links, compare two designs	P1	A, B
F-25	Leaderboard (local P1, online P2) + challenge codes (P1)	P1/P2	A, B
F-26	Mission Academy glossary + "?" tooltips everywhere	P1	L, B
F-27	Guided tutorial on first Cadet mission	P1	L, B
F-28	Diverse crew, avatar select, portraits that react (neutral/happy/worried)	P0	C, B, L
F-29	Settings: volume, mute, text size, reduced motion	P1	B
F-30	Sandbox mode	P1	A, B
F-31	Agency career (Tycoon)	P2	A, B, L
F-32	In-room chat (short messages)	P1	A, B
F-33	Mission Twin: the real mission closest to your design (after launch + in report)	P1	L (data), A, B
F-34	Trade-Space Explorer: science, cost and margins vs one chosen parameter	P1	A, B
F-35	Concept Maturity meter (CML 2/3/4, green/yellow/red, 80% rule) in the review	P0	A, B

ID	Feature	Pri	Owners
F-36	Requirements panel with "Verified by" column; user segment line	P0	B, L

A6. Screens, routes and flow

The app is a single-page web app with a hash router (works on GitHub Pages without server config). Logical canvas is 16:9; target screens 1280×720 up to 1920×1080; must work at 1366×768.

Route	Screen	Notes
`#/`	Title	Play Solo · Create Team Room · Join Room · Mission Academy · Saved & Compare · Leaderboard · Settings · Credits
`#/solo`	Solo setup	Scenario cards, difficulty, avatar, optional challenge code → Start
`#/room/new`	Create room	Your name + avatar → creates code → goes to lobby
`#/join` and `#/join/:code`	Join room	Enter code (or prefilled), name, avatar
`#/room/:code`	Room shell	Renders the current phase: lobby → council → design → launch → ops → report
`#/play`	Solo shell	Same phase screens, driven by LocalRoom
`#/academy`	Mission Academy	Glossary cards (F-26)
`#/saved`	Saved designs + compare	(F-24)
`#/design/:code`	Open shared design	Decodes and opens it in solo shell
`#/leaderboard`	Leaderboard	Filter by scenario/difficulty/challenge code
`#/settings`, `#/credits`	Settings, credits	Credits list every asset + tool

Phase flow (both solo and team): `lobby` → `council` → `design` (review is a panel inside design) → `launch` → `ops` → `report`. In solo, `lobby` is skipped and `council` is a single confirm screen.

Phase stepper labels (NASA process, from Levri 2025) shown in the top bar: **Need & Goals** (briefing) → **Concept** (council) → **Requirements & Design** (design) → **Build & Verify** (review + launch) → **Implement** (ops) → **Results** (report). A small CML badge sits under Need & Goals (CML 1), Design (CML 2–3) and Build & Verify (CML 4, or the level actually reached).

A7. Game roles and team mechanics

Four **game roles** (separate from the six team-member jobs). Every mission always has all four. A player may hold several roles (solo player holds all four).

Role ID	Title	Owns (design fields)	Owns (events)	Role goal (counts 15 pts each)
`planner`	Mission Commander	orbit, mission lifetime, launch vehicle, delta-v margin	budget_cut, debris_conjunction	Total cost ≤ 90% of budget AND launch on time (launchMonth ≤ window)
`engineer`	Spacecraft Engineer	bus, solar array (+W), RTGs, battery (+Wh), computer, attitude control, propulsion	solar_storm, micrometeoroid, wheel_failure, battery_aging	Launch mass margin ≥ 10% AND power margin ≥ 15% AND battery covers eclipse
`science`	Science Lead	instruments	discovery, dust_storm	Science returned ≥ 100% of scenario target
`comms`	Communications & Data Officer	radio, high-gain antenna, ground network	ground_outage, computer_reset	≥ 95% of generated data returned to Earth

Mission segments (Levri 2025): Commander = launch segment (rocket, orbit, schedule) · Engineer and Science Lead = space segment (spacecraft and instruments) · Comms Officer = ground segment (radio, antennas, ground stations) · the **user segment** is the people who use the data, named on each scenario and in the report. The four roles cover every segment, so the team designs the *whole* mission, not just a spacecraft.

The goals pull against each other on purpose: Science wants more instruments; Engineer wants margins; Comms wants downlink headroom; Commander wants money left over. That is the negotiation the challenge asks for.

Budget Council (F-20): before design, the budget is split in percent between the four roles. Default split: planner 35, engineer 30, science 25, comms 10. Any player can drag the sliders (they always sum to 100); each player then presses **Agree**. Design opens when every connected player has agreed. Any slider change clears all agrees.

- Each role's **cost** (A9) is compared with its **allocation** = budget × split%. Going over allocation is allowed but shows a red "over allocation" badge on that role tab and a `warn` check.
- **Funds request:** a role can request an amount (in $M, step 0.5) from another role with a one-line reason. The receiving role's owner accepts or declines. Accepting moves that many percentage points of allocation. Requests appear as toasts and in a requests list.

Sign-offs (F-21): each role has a sign-off toggle in the review panel. **Any design change by anyone clears all four sign-offs** and shows a toast ("Science changed instruments — all sign-offs reset"). The host can press **LAUNCH** only when all four are signed. If any `fail` check exists, a confirmation dialog warns "NO-GO items present — launching will likely fail. Launch anyway?" Launching with failures is allowed on purpose: learning from failed missions is part of the challenge.

Event ownership: during operations each event belongs to one role. The player holding that role chooses an option; other players can tap an option to *suggest* it (suggestions appear as small avatars next to options). In team mode a timer runs (difficulty table); when it expires, the event's default option is applied.

A8. Game data (exact values)

All values are **simplified game values inspired by public data**, tuned for play. Units: mass kg, power W, money $M (millions of US dollars), data Gbit (gigabits), delta-v m/s, time months/years. The Lead turns these tables into JSON files in `src/data/` (field names in A14). Tier `basic` = always available; `advanced` = locked on Cadet difficulty.

A8.1 Orbits (`src/data/orbits.json`)

field	leo	sso	geo	lunar	l2	mars
name	Low Earth Orbit	Sun-Synchronous Orbit	Geostationary Orbit	Low Lunar Polar Orbit	Sun-Earth L2 Halo Orbit	Mars Polar Orbit
destination (rocket target)	LEO	SSO	GTO	TLI	L2	TMI
altitude km	500	700	35786	100	1500000	300
period min	95	99	1436	118	259200	113
sunFraction	0.62	0.65	0.99	0.60	1.0	0.65
eclipseMin	36	35	72	47	0	40
solarFactor	1.0	1.0	1.0	1.0	1.0	0.43
insertionDvMs	0	0	1500	900	50	1100
stationKeepingDvMsPerYear	25	10	50	100	3	15
disposalDvMs	60	150	11	0	1	20
naturalLifetimeYears (no propulsion)	4	25	999	0.5	0.1	50
commsFactor	1.0	1.0	0.05	0.002	0.0003	0.00002
lightDelaySec	0	0	0.12	1.3	5	750
radiation	1.0	1.2	2.0	2.5	2.5	3.0
opsCostMPerYear	2	2	2	4	4	4
cruiseMonths	0	0	0.1	0.2	1	7
magnetorquersWork	yes	yes	no	no	no	no
isNearEarth	yes	yes	yes	no	no	no
debrisRule	yes	yes	no	no	no	no

Contact hours per day (`contactHours`, by ground network; 0 = cannot reach):

orbit	campus	commercial	deepspace
leo	0.5	2.0	1.0
sso	0.4	3.0	1.0
geo	24	24	24
lunar	0	0	8
l2	0	0	4
mars	0	0	6

Player-facing explanations (shown in the orbit picker; Lead may polish wording):

- **leo** — 500 km up, circles Earth every 95 minutes. Close = sharp images and cheap launches. But you only see a ground station for a few minutes per pass, you spend 36 min of every orbit in Earth's shadow, and thin air slowly drags you down, so you need fuel to stay up and to deorbit at the end (space-debris rule).
- **sso** — a polar orbit tilted so you pass over every place at the same local time: perfect lighting for comparing images over months. Passes over polar ground stations often. Needs fuel to deorbit at end of life.
- **geo** — 35,786 km up; one orbit takes exactly one day, so you hover over the same spot — continuous weather watching and 24-hour contact with one station. Far away: blurry for small details, weaker radio signal, the rocket drops you in a transfer orbit and your spacecraft must burn 1,500 m/s to circularize.
- **lunar** — 100 km over the Moon's poles: maps the whole surface, including dark craters near the poles. Needs a deep-space antenna network; the Moon's lumpy gravity disturbs the orbit, so station-keeping fuel is high; no magnetic field, so magnetorquers do not work.
- **l2** — 1.5 million km from Earth on the side away from the Sun. The Sun, Earth and Moon are always behind you, so a telescope stays cold and steady with no eclipses. Far: deep-space network required, limited contact hours. The orbit is unstable, so small corrections are needed.
- **mars** — sunlight at Mars is only 43% as strong, so solar arrays must be larger. Radio signals take minutes to arrive (average 12.5 min one way), so the spacecraft must handle emergencies on its own. Getting there takes ~7 months and Mars launch windows open only every 26 months.

A8.2 Ground networks (`groundNetworks` in `src/data/components.json`)

id	name	gain	cost $M/yr	description
campus	University Ground Station (Cairo, 3 m dish)	1	0.5	One small dish. Cheap, few minutes per pass, cannot reach beyond Earth orbit.
commercial	Commercial Polar Network (several 5–7 m dishes)	3	3	Stations near the poles see polar orbits on almost every pass.
deepspace	Deep-Space Network (34–70 m dishes on 3 continents)	30	8	Huge dishes that can hear spacecraft at the Moon, L2 and Mars. Time is shared, so you get scheduled hours.

A8.3 Launch vehicles (`src/data/launchVehicles.json`)

Capacity in kg delivered to each destination. Missing destination = cannot go there.

id	name	class note	LEO	SSO	GTO	TLI	L2	TMI	cost $M	reliability	tier
spark	Spark	small launcher	300	200	–	–	–	–	8	0.93	basic
rideshare	Rideshare Slot	shared ride on a medium rocket, SSO only, max 300 kg	–	300	–	–	–	–	0.0065 × kg (min 0.5)	0.97	basic
arrow	Arrow	light-medium launcher	2300	1500	900	700	650	400	40	0.94	basic
workhorse	Workhorse	reusable medium-lift	17500	13000	5500	3500	3000	2500	70	0.98	basic
titan	Titan	heavy-lift	60000	45000	22000	15000	13000	12000	150	0.96	advanced

Rideshare: `costPerKgM` = 0.0065, `minCostM` = 0.5, `scheduleIntervalMonths` = 4 (launches only in months that are multiples of 4 — you may wait).

A8.4 Spacecraft components (`src/data/components.json`)

Buses (structure, thermal, wiring):

id	name	mass	cost	base load W	instrument slots	max total mass	spare wheel	tier
bus_cube	CubeSat 12U Bus	6	1	8	1	25	no	basic
bus_small	Small Bus	60	8	40	2	300	no	basic
bus_medium	Medium Bus	250	25	120	4	2000	yes	basic
bus_large	Large Bus	800	70	300	6	8000	yes	advanced

Solar arrays (player picks type and rated watts, slider 0–6000 W, step 50; `none` = no array):

id	name	W per kg	cost $M per W	degradation per year	tier
sa_si	Silicon Solar Array	25	0.001	0.03	basic
sa_gaas	Triple-Junction GaAs Array	60	0.002	0.015	advanced

RTG (count 0–3): `rtg` Radioisotope Power Unit — 45 kg, 110 W each, $60M each, decays 4%/yr, works in shadow and dust, tier advanced.

Batteries (type + capacity slider 50–5000 Wh, step 50):

id	name	Wh per kg	cost $M per Wh	usable depth of discharge	tier
bat_liion	Li-ion Battery	150	0.0005	0.4	basic
bat_liion_adv	High-Energy Li-ion Battery	250	0.001	0.5	advanced

Computers:

id	name	mass	power	cost	storage Gbit	compression	autonomy	rad-hard	tier
obc_basic	Standard Flight Computer	1.5	7	0.5	512	1.0	no	no	basic
obc_adv	Rad-Hard Computer with Compression	5	20	3	4096	0.5	yes	yes	advanced

Compression 0.5 halves the data volume without losing science. Autonomy lets the spacecraft react to emergencies on its own (needed when light delay > 60 s).

Attitude control (ADCS):

id	name	mass	power	cost	pointing accuracy (deg)	near-Earth only	tier
adcs_mag	Magnetorquers + Sun Sensors	2	3	0.3	5	yes	basic
adcs_wheels	Reaction Wheels + Star Tracker	15	40	4	0.05	no	basic
adcs_precision	Precision Pointing System	35	70	12	0.001	no	advanced

Propulsion (propellant is computed automatically from delta-v — see A9):

id	name	dry mass	Isp s	cost	power W	max propellant kg	electric	tier
prop_none	No Propulsion	0	0	0	0	–	no	basic
prop_coldgas	Cold-Gas Thrusters	5	65	0.5	0	20	no	basic
prop_mono	Hydrazine Monopropellant	20	220	3	10	–	no	basic
prop_biprop	Bipropellant Engine	60	315	8	20	–	no	basic
prop_electric	Electric Hall Thrusters	40	1600	12	10	–	yes	advanced

Electric propulsion needs `solarArrayW × solarFactor ≥ 1000` and adds `insertionDv / 250` months of transfer time.

Radios:

id	name	mass	power W (while transmitting)	cost	base rate Mbps	max rate Mbps	near-Earth only	rain fade	tier
radio_uhf	UHF Radio	1	10	0.2	0.0096	0.0096	yes	no	basic
radio_sband	S-Band Transmitter	3	20	1	2	10	no	no	basic
radio_xband	X-Band Transmitter	6	45	3	150	800	no	no	basic
radio_kaband	Ka-Band Transmitter	8	70	6	600	2000	no	yes	advanced

High-gain antenna (toggle): `hga` High-Gain Dish Antenna (1.5 m) — 20 kg, 0 W, $6M, gain ×20, works only if ADCS pointing ≤ 0.1°, tier basic.

A8.5 Instruments (`src/data/instruments.json`)

id	name	mass	power	cost	data Gbit/day	pointing need deg	tags	base science /yr	dev months	tier
cam_wide	Wide-Field Camera	5	10	2	20	1.0	optical	10	6	basic
cam_hires	High-Resolution Telescope Camera	40	60	25	200	0.05	optical_hr, optical	30	18	advanced
imager_ms	Multispectral Imager	25	40	15	80	0.1	multispectral	25	12	basic
imager_tir	Thermal Infrared Imager	20	50	12	40	0.1	thermal_ir	20	12	basic
radar_sar	Synthetic Aperture Radar	150	400	60	400	0.1	radar	40	24	advanced
sounder_atm	Atmospheric Sounder	60	120	30	30	0.5	sounder	30	18	basic
lightning_map	Lightning Mapper	40	70	20	50	0.2	lightning	20	15	basic
spec_neutron	Neutron Spectrometer	12	10	8	0.5	5.0	neutron	25	12	basic
lidar_alt	Laser Altimeter	15	35	10	5	0.05	altimeter	20	12	basic
magnetometer	Magnetometer	3	4	1.5	0.1	10	magnetic	10	6	basic
spec_atm	Atmospheric Spectrometer	20	30	14	15	0.1	spectrometer	25	15	basic

id	name	mass	power	cost	data Gbit/day	pointing need deg	tags	base science /yr	dev months	tier
det_particle	Radiation & Particle Detector	4	5	2	0.2	20	particles	8	6	basic
relay_uhf	Rover Relay Radio	30	100	15	2	5	relay	30	12	basic
tel_ir	Cryogenic Infrared Telescope	400	300	250	60	0.001	telescope_ir	80	30	advanced

`det_particle` has `special: "particle_warning"` (halves solar-storm damage risk). Plain-language "what it measures" text for each instrument is in the Lead's brief and goes into the JSON `whatItMeasures` field.

A8.6 Scenarios (`src/data/scenarios.json`)

id	name	default difficulty	required tag groups (all groups needed; any tag inside a group)	orbit fit	min life yrs	budget $M	launch window months	hard window	science target	events
nile_watch	Eyes on the Nile	cadet	[multispectral, optical, optical_hr]	sso 1.0, leo 0.8	2	120	36	no	110	3
storm_watch	Storm Watcher	engineer	[sounder, thermal_ir]	geo 1.0, sso 0.4, leo 0.3	5	320	36	no	300	4
ice_hunter	Ice Hunter	engineer	[neutron] + [altimeter, thermal_ir]	lunar 1.0	1	250	30	no	80	4
red_relay	Red Relay	director	[spectrometer] + [relay]	mars 1.0	2	300	26	yes	110	5
first_light	First Light	director	[telescope_ir]	l2 1.0, geo 0.5, leo 0.3	5	650	48	no	360	4
sandbox	Sandbox	engineer	none	all orbits 1.0	1	2000	120	no	100	3

Tag weights (science multiplier per instrument = highest weight among its tags; tags not listed use `defaultTagWeight` = 0.1):

- nile_watch: multispectral 1.0, radar 0.9, thermal_ir 0.8, optical_hr 0.7, optical 0.5
- storm_watch: sounder 1.0, thermal_ir 0.9, lightning 0.8, optical 0.4, particles 0.3
- ice_hunter: neutron 1.0, altimeter 0.9, thermal_ir 0.8, optical_hr 0.7, optical 0.6, particles 0.4, magnetic 0.3
- red_relay: spectrometer 1.0, relay 1.0, thermal_ir 0.7, optical_hr 0.6, optical 0.5, particles 0.4, magnetic 0.4
- first_light: telescope_ir 1.0, spectrometer 0.3, particles 0.2
- sandbox: defaultTagWeight 1.0

Event pools:

- nile_watch: solar_storm, ground_outage, debris_conjunction, discovery, micrometeoroid
- storm_watch: solar_storm, wheel_failure, budget_cut, computer_reset, discovery, battery_aging
- ice_hunter: solar_storm, wheel_failure, computer_reset, discovery, budget_cut, battery_aging
- red_relay: dust_storm, solar_storm, computer_reset, wheel_failure, discovery, budget_cut
- first_light: solar_storm, micrometeoroid, wheel_failure, computer_reset, discovery, budget_cut
- sandbox: all ten

Recommended orbit: nile_watch sso, storm_watch geo, ice_hunter lunar, red_relay mars, first_light l2, sandbox leo.

User segment ("Who uses your data", field `userSegment`): nile_watch: farmers, irrigation engineers and water ministries across the Nile Delta · storm_watch: weather forecasters, civil-protection agencies and airlines · ice_hunter: mission planners choosing safe, water-rich landing sites for future crews · red_relay: planetary scientists and the rover teams whose data you relay · first_light: astronomers studying the first galaxies and the atmospheres of distant planets · sandbox: you.

Mission Twins (field `twins`, F-33). Real missions to compare with. Matching: for each twin in the scenario's list, score = 2 if `orbit` equals the design's orbit, + 1 per tag shared between `keyTags` and the design's instrument tags; the highest score wins (ties: list order). Sandbox uses every twin from all scenarios.

scenario	name	agency	launched	orbit	keyTags	fact (shown in the card)
nile_watch	Landsat 9	NASA / USGS	2021	sso	multispectral, thermal_ir	Images the whole Earth in visible, infrared and thermal bands; used worldwide to track crops and water.

scenario	name	agency	launched	orbit	keyTags	fact (shown in the card)
nile_watch	Sentinel-2	ESA	2015	sso	multispectral	A pair of satellites revisiting every 5 days with 13 colour bands for vegetation monitoring.
storm_watch	GOES-16	NASA / NOAA	2016	geo	thermal_ir, lightning	Watches the Americas from geostationary orbit; carried the first lightning mapper in GEO.
storm_watch	Meteosat Third Generation Imager-1	ESA / EUMETSAT	2022	geo	thermal_ir, lightning	Watches Europe and Africa from GEO with an imager and a lightning imager.
ice_hunter	Lunar Prospector	NASA	1998	lunar	neutron	Its neutron spectrometer found signs of hydrogen, a hint of ice, at the lunar poles.
ice_hunter	Lunar Reconnaissance Orbiter	NASA	2009	lunar	altimeter, thermal_ir, neutron	Maps the Moon with a laser altimeter, a thermal radiometer and a neutron detector.
red_relay	Mars Reconnaissance Orbiter	NASA	2005	mars	relay, optical_hr, spectrometer	Studies Mars with a high-resolution camera and spectrometers, and relays rover data to Earth.
red_relay	ExoMars Trace Gas Orbiter	ESA / Roscosmos	2016	mars	spectrometer, relay	Hunts for methane in the Martian air and relays data for surface missions.
red_relay	MAVEN	NASA	2013	mars	spectrometer, particles, magnetic, relay	Studies how Mars lost its atmosphere, and also relays rover data.
first_light	James Webb Space Telescope	NASA / ESA / CSA	2021	l2	telescope_ir	A 6.5 m gold-coated infrared telescope at L2, shaded by a tennis-court-size sunshield.
first_light	Herschel Space Observatory	ESA	2009	l2	telescope_ir	A far-infrared telescope at L2 that studied how stars and galaxies form.

Story (one-line objective, shown on the scenario card):

- **Eyes on the Nile** — Monitor crops and water across the Nile Delta so farmers can save water and spot crop stress early.
- **Storm Watcher** — Watch dust storms and flash floods over Africa and the Middle East around the clock.
- **Ice Hunter** — Map water ice in the Moon's permanently shadowed craters to choose landing sites for future crews.
- **Red Relay** — Study methane and dust in the Martian atmosphere while relaying data home for rovers on the surface.
- **First Light** — Fly an infrared telescope to see the first galaxies and the atmospheres of distant planets.

A8.7 Difficulties (`src/data/difficulties.json`)

id	name	budget mult	event mult	launch failures	advanced unlocked	hints	decision timer s
cadet	Cadet	1.2	0.5	no	no	yes	60
engineer	Engineer	1.0	1.0	yes	yes	no	40
director	Flight Director	0.9	1.5	yes	yes	no	25

Number of events = `max(1, round(scenario.eventCount × eventMult))`. Hints = "suggested" badges on recommended choices plus Dr. Hassan tips.

A8.8 Random events (`src/data/events.json`)

Effect types (implemented by Programmer A): `loseScienceDays(days)` instruments off; `scienceMult(value, days?)` multiply science (no days = permanent); `powerMult(value, days?)`; `contactLossDays(days)` no downlink; `costM(value)` adds operations cost; `deltaVUse(value)` uses propellant (m/s); `scienceBonusPct(value)` adds value% of one year's planned science; `pointingMult(value)` multiplies ADCS pointing error permanently; `lifetimeYearsDelta(value)`; `dataMult(value, days)` multiplies data generated; `endMission(reason)`; `downgradeGround()` switches ground network one step cheaper; `immunity(eventId)` that event cannot happen again.

id	title	owner	conditions	options (✓ = safe/good choice)	default
solar_storm	Solar Storm!	engineer	any	**safe_mode** ✓ loseScienceDays 3; requires autonomy if light delay > 60 s · **keep_observing** risk 0.25 → scienceMult 0.7 permanent ("an instrument was damaged"); risk ×0.5 if `det_particle` carried, ×0.5 if `obc_adv`	safe_mode
micrometeoroid	Micrometeoroid Strike	engineer	any	auto: powerMult 0.9 permanent · **reduce_load** ✓ scienceMult 0.85 permanent · **full_ops** risk (0.6 if power margin after hit < 0, else 0.05) → loseScienceDays 10	reduce_load
ground_outage	Ground Station Outage	comms	any (twice as likely if `radio_kaband`)	**rent_backup** ✓ costM 1.5 · **wait** contactLossDays 3	wait
wheel_failure	Reaction Wheel Failure	engineer	ADCS is adcs_wheels or adcs_precision	**spare_wheel** ✓ nothing lost; requires bus with spare wheel · **magnetorquer_mode** pointingMult 5; requires near-Earth orbit · **accept** pointingMult 20	accept
budget_cut	Budget Cut!	planner	any	**pay_reserve** ✓ costM = 10% of budget; requires reserve ≥ that amount · **shorten** lifetimeYearsDelta −1 · **cheaper_ground** downgradeGround	shorten
computer_reset	Computer Reset	comms	any (auto-handled with `obc_adv`: info only, loseScienceDays 0)	**reload** loseScienceDays 1 + contactLossDays 1 · **patch** ✓ costM 0.5 + loseScienceDays 2 + immunity computer_reset	reload
debris_conjunction	Space Debris Close Approach	planner	orbits leo, sso, geo	**avoid** ✓ deltaVUse 2 + loseScienceDays 1; requires propulsion · **stay** risk 0.02 → endMission "collision with debris"	avoid if possible, else stay
discovery	Unexpected Discovery!	science	any	**follow_up** ✓ scienceBonusPct 15 + dataMult 3 for 5 days · **stick_to_plan** nothing	stick_to_plan
dust_storm	Global Dust Storm on Mars	science	orbit mars	**observe** ✓ scienceBonusPct 25 + relay science ×0.5 for 30 days · **relay_focus** scienceBonusPct 5	relay_focus
battery_aging	Battery Wearing Out	engineer	any orbit with eclipse > 0	auto: battery capacity ×0.8 permanent · **eclipse_saving** ✓ scienceMult 0.9 permanent · **push_on** risk (0.3 if battery now below requirement, else 0.02) → loseScienceDays 15	eclipse_saving

Every option has an `explanation` sentence shown after choosing (written by the Lead): what happened and why, in plain words.

A9. Formulas (Programmer A implements exactly; QA verifies with A17)

Constants (`src/data/constants.json`): `G0 = 9.81`, `SOLAR_EFFICIENCY = 0.85`, `INTEGRATION_OVERHEAD = 0.20`, `TANK_FRACTION = 0.10`, `RADIO_STANDBY_W = 2`, `MIN_BATTERY_WH = 50`, `HGA_MAX_POINTING_DEG = 0.1`, `DELAY_COST_M_PER_MONTH = 1.5`, `BASE_DEV_MONTHS = 12`, `ADVANCED_DEV_MONTHS = 2`, `MARS_SYNODIC_MONTHS = 26`, `ELECTRIC_MIN_W = 1000`, `ELECTRIC_DV_PER_MONTH = 250`, `MAX_LIFETIME_YEARS = 10`, `DAYS_PER_YEAR = 365`.

Mass

```
dryKg = bus.massKg + Σ instrument.massKg
      + (solarArrayW / array.wPerKg)            [0 if no array]
      + rtgCount × rtg.massKg
      + batteryWh / battery.whPerKg
      + computer.massKg + adcs.massKg + propulsion.dryMassKg
      + radio.massKg + (hga ? hga.massKg : 0)

dvRequired   = orbit.insertionDvMs + orbit.stationKeepingDvMsPerYear × lifetimeYears + orbit.disposalDvMs
dvWithMargin = dvRequired × (1 + deltaVMarginPct / 100)
x            = exp(dvWithMargin / (propulsion.ispS × G0)) − 1        [x = 0 if prop_none]
propellantKg = dryKg × x / (1 − TANK_FRACTION × x)                 [if denominator ≤ 0.05 → infeasible]
if propulsion.maxPropellantKg and propellantKg > max: propellantKg = max (capability reduced)
tankKg   = TANK_FRACTION × propellantKg
totalKg  = dryKg + propellantKg + tankKg
dvCapability = ispS × G0 × ln((dryKg + tankKg + propellantKg) / (dryKg + tankKg))
launchCapacityKg = vehicle.capacityKg[orbit.destination]  (undefined → cannot reach)
launchMarginPct  = (launchCapacityKg − totalKg) / launchCapacityKg × 100
```

Communications and data

```
contactHours   = orbit.contactHours[groundNetworkId]
hgaEffective   = hga AND adcs.pointingDeg ≤ HGA_MAX_POINTING_DEG
rateMbps       = min(radio.maxRateMbps,
                     radio.baseRateMbps × orbit.commsFactor × ground.gain × (hgaEffective ? hga.gain : 1))
downlinkGbitPerDay  = rateMbps × contactHours × 3.6
generatedGbitPerDay = Σ instrument.dataGbitPerDay × computer.compression
returnPct = generated = 0 ? 100 : min(100, downlink / generated × 100)
```

Power

```
arrayAvgW   = solarArrayW × orbit.solarFactor × orbit.sunFraction × SOLAR_EFFICIENCY
rtgW        = rtgCount × rtg.powerW
generationW = arrayAvgW + rtgW
loadW = bus.baseLoadW + Σ instrument.powerW + computer.powerW + adcs.powerW + propulsion.powerW
      + radio.powerW × min(1, contactHours / 24) + RADIO_STANDBY_W
powerMarginPct    = (generationW − loadW) / loadW × 100
batteryRequiredWh = max(MIN_BATTERY_WH, max(0, loadW − rtgW) × orbit.eclipseMin / 60 / battery.depthOfDischarge)
batteryOk         = batteryWh ≥ batteryRequiredWh
```

Pointing quality per instrument: `quality = clamp(instrument.pointingReqDeg / adcs.pointingDeg, 0.1, 1)`.

Cost (all $M)

```
hwEngineer = bus.costM + solarArrayW × array.costMPerW + rtgCount × rtg.costM + batteryWh × battery.costMPerWh
           + computer.costM + adcs.costM + propulsion.costM
hwScience  = Σ instrument.costM
hwComms    = radio.costM + (hga ? hga.costM : 0)
launchCost = vehicle.costM, or for rideshare max(minCostM, costPerKgM × totalKg)
cost.engineer = hwEngineer × (1 + INTEGRATION_OVERHEAD)
cost.science  = hwScience  × (1 + INTEGRATION_OVERHEAD)
cost.comms    = hwComms    × (1 + INTEGRATION_OVERHEAD) + ground.costMPerYear × lifetimeYears
cost.planner  = launchCost + orbit.opsCostMPerYear × lifetimeYears + delayCostM
totalCost  = Σ cost.role
budgetM    = scenario.budgetM × difficulty.budgetMult
reservePct = (budgetM − totalCost) / budgetM × 100
allocationM[role] = budgetM × split[role] / 100
```

Time

```
advancedCount  = number of selected components/instruments with tier "advanced"
devMonths      = BASE_DEV_MONTHS + max(instrument.devMonths, 0) + ADVANCED_DEV_MONTHS × advancedCount
launchMonth    = devMonths, rounded UP to a multiple of 4 if vehicle is rideshare
if scenario.hardLaunchWindow:                        (Mars)
   window = launchWindowMonths
   while launchMonth > window: window += MARS_SYNODIC_MONTHS
   launchMonth = (window == launchWindowMonths) ? launchMonth : window   (wait for the next window)
delayMonths    = max(0, launchMonth − launchWindowMonths)
delayCostM     = DELAY_COST_M_PER_MONTH × delayMonths
transferMonths = orbit.cruiseMonths + (electric ? orbit.insertionDvMs / ELECTRIC_DV_PER_MONTH : 0)
```

Example: Mars window at month 26, development finishes at month 30 → launch at month 52 → delay 26 months → $39M extra and the Commander's role goal fails.

A10. Readiness checks (runChecks)

Each check returns `{ id, severity, role, title, message, fixHint, conceptId }`. Messages explain **why** in plain language with numbers inserted. Exact English text is written by the Lead in `src/data/checks.json` (keys = check id; `{placeholders}` filled by code).

id	role	fail when	warn when	concept
launch_destination	planner	vehicle has no capacity for orbit destination (or rideshare not SSO)	–	payload_capacity
launch_capacity	planner	totalKg > launchCapacityKg	margin < 10%	payload_capacity
bus_mass	engineer	totalKg > bus.maxMassKg	totalKg > 90% of max	mass_budget
bus_slots	science	instruments > bus.instrumentSlots	–	mass_budget
power_balance	engineer	generationW < loadW	margin < 15%	power_budget
battery_eclipse	engineer	batteryWh < 50% of required	batteryWh < required	eclipse
deltav_capability	engineer	dvCapability < orbit.insertionDvMs, or rocket equation infeasible	dvCapability < dvRequired (lifetime will be cut)	delta_v
electric_power	engineer	electric propulsion and solarArrayW × solarFactor < 1000	–	delta_v
adcs_environment	engineer	adcs.nearEarthOnly and orbit not near Earth	–	pointing
pointing_quality	science	–	any instrument quality < 1 (list them)	pointing
hga_pointing	comms	–	hga selected but not effective	antenna_gain
radio_range	comms	radio.nearEarthOnly and orbit not near Earth, or contactHours = 0 for chosen network	–	ground_station
data_return	comms	returnPct < 25	returnPct < 95	data_rate
storage	comms	–	daily excess (generated − downlink) × 3 > storage	data_rate
budget_total	planner	totalCost > budgetM	reservePct < 5	mission_cost
role_allocation	(each role)	–	cost.role > allocationM.role	mission_cost
launch_window	planner	–	launchMonth > window (show delay months and cost)	launch_window
required_instruments	science	any required tag group not satisfied	–	mission_requirements
orbit_fit	planner	orbit not in scenario.orbitFit	fit < 1	orbit
lifetime_min	planner	–	lifetimeYears < scenario.minLifetimeYears	mission_lifetime
debris_disposal	planner	–	orbit.debrisRule and (prop_none or dvCapability < dvRequired)	space_debris
natural_lifetime	planner	–	prop_none and lifetimeYears > orbit.naturalLifetimeYears	orbit

Sort: fail first, then warn, then pass; group by role in the UI.

Concept Maturity meter (F-35, from JPL's CML paper). Treat each check as a checklist item: pass = green, warn = yellow, fail = red. `greenPct = passCount / totalChecks × 100`. Level reached: **CML 4 "Point Design"** if greenPct ≥ 80 and no red; else **CML 3 "Trade Space"** if greenPct ≥ 50; else **CML 2 "Initial Feasibility"**. The review header shows: "Concept Maturity: CML 3 Trade Space · 72% green · 4 yellow · 1 red — reach 80% green with no red for CML 4 Point Design". Launching below CML 4 is allowed (see A7), but the NO-GO dialog names the level reached.

Template variables. `runChecks` passes **all** of these to **every** template, so any text may use any of them. Numbers have at most 1 decimal; money is pre-formatted like `$70.8M`; percentages are plain numbers (the template writes the `%`).

```
{totalKg} {capKg} {launchMargin} {vehicle} {destination} {busName} {busMaxKg} {slots}
{instrCount} {genW} {loadW} {powerMargin} {batteryWh} {batteryReqWh} {eclipseMin}
{dvRequired} {dvCapability} {insertionDv} {rateMbps} {downlink} {generated} {returnPct}
{storage} {totalCost} {budget} {reservePct} {role} {roleCost} {roleAlloc} {launchMonth}
{window} {delayMonths} {delayCost} {lifetime} {minLifetime} {orbit} {fit} {recommendedOrbit}
```

`{missingTags}` `{blurredList}` `{adcsName}` `{adcsDeg}` `{radio}` `{ground}` `{disposalDv}` `{naturalLife}` `{lightDelay}`

`{role}` is the role title (e.g. "Science Lead") of the check; `{missingTags}` lists missing groups as words ("neutron, or altimeter / thermal infrared"); `{blurredList}` lists instruments with quality < 1 as "Laser Altimeter (20%)"; `{fit}` is a percentage; `{lightDelay}` is formatted ("12.5 min").

A11. Mission simulation (`createSimulation`)

Deterministic: same inputs + same seed + same decisions = same result on every device. Random numbers come only from a seeded PRNG (mulberry32) created from `seed`.

Before day 0:

1. **Launch:** if `totalKg > launchCapacityKg` or destination unreachable → outcome `launch_failure` (reason: "too heavy for the rocket" / "rocket cannot reach this orbit"). Else if difficulty has launch failures and `rng() < vehicle.reliability` → `launch_failure` (reason: "launch vehicle anomaly"). Cadet never has random launch failure.
2. **Orbit insertion:** if `dvCapability < orbit.insertionDvMs` → outcome `orbit_failure`.
3. **Event schedule:** count = events per difficulty (A8.7). Filter the pool by conditions. Draw events without repetition (repeat allowed only if the pool is smaller than the count). Assign each a day = `floor(rng() × plannedDays × 0.9) + 1`, sorted ascending. Draw each event's risk roll in advance in schedule order, so every client gets the same rolls.
4. `plannedDays = round(lifetimeYears × 365)`. `endDay` = minimum of planned days, natural lifetime days (if prop_none), and the day propellant for station-keeping runs out (dvCapability − insertion − disposal reserve, spent at stationKeepingDvMsPerYear / 365 per day).

Each day d (1 … endDay):

```
years       = d / 365
arrayW_t    = solarArrayW × (1 − array.degradationPerYear)^years
rtgW_t      = rtgCount × rtg.powerW × (1 − rtg.decayPerYear)^years
generation  = arrayW_t × solarFactor × sunFraction × SOLAR_EFFICIENCY × powerMult_active + rtgW_t
nonInstrumentLoad = loadW − Σ instrument.powerW
if generation < nonInstrumentLoad → outcome "power_failure", stop (computer and radio cannot run)
duty        = clamp((generation − nonInstrumentLoad) / Σ instrument.powerW, 0, 1)
duty       *= batteryOk_t ? 1 : max(0.2, batteryWh_t / batteryRequiredWh)
duty        = 0 on days covered by loseScienceDays
generatedToday = generatedGbitPerDay × duty × dataMult_active
capacity    = downlinkGbitPerDay  (0 during contactLossDays)
queue       = stored + generatedToday;  sent = min(queue, capacity);  stored = queue − sent
if stored > computer.storageGbit: lost += stored − storage; stored = storage
scienceGeneratedToday = Σ_i baseScience_i × tagWeight_i × orbitFit × quality_i × duty × scienceMult_active / 365
   (relay instruments use relayMult during dust storm)
totals: dataGenerated += generatedToday; dataReturned += sent; scienceGenerated += scienceGeneratedToday
```

Science returned = `scienceGenerated × (dataReturned / dataGenerated)` (1 if no data generated) + bonuses from `scienceBonusPct` (value% × one year of planned science at full duty).

Event day: the stepper stops with `pendingEvent`. Auto-effects apply first. If the computer auto-handles it (computer_reset with obc_adv), there is no decision. Otherwise wait for `decide(optionId)`. Options whose `requires` is not met are shown disabled with the reason. Apply the option's effects; if it has a risk, use the pre-drawn roll for this event: `roll < chance × Π(reductions)` → apply risk effects and mark the event outcome **bad**. Log the event, the chosen option, who chose it, and the result text.

Samples for charts: record a sample every `max(1, floor(endDay / 200))` days: `{ day, generationW, loadW, duty, stored, dataGeneratedCum, dataReturnedCum, scienceCum, costCum }`.

Outcomes: `launch_failure`, `orbit_failure`, `power_failure`, `ended_early` (propellant, natural decay, collision or budget shortened below the minimum), `completed`.

A12. Scoring (`scoreMission`) — maximum 1000

Part	Max	Rule
Objectives	300	150 × (satisfied required groups / total groups; 1 if none) + 50 × orbitFit + 100 × min(1, achievedYears / minLifetimeYears)
Science	300	300 × min(1, scienceReturned / scienceTarget)

Part	Max	Rule
Efficiency	200	Budget 100 × clamp(finalReservePct / 15, 0, 1) (0 if over budget) · Mass margin 50: m<0 → 0; 0–5 → linear 0→50; 5–60 → 50; 60–95 → linear 50→30; ≥95 → 30 · Power margin 50: p<0 → 0; 0–10 → linear 0→50; 10–60 → 50; 60–150 → linear 50→25; ≥150 → 25
Operations	100	100 − 40 × dataLostFraction − 15 × (bad event outcomes) − 20 × (debris rule violated) − 30 × (power_failure); min 0
Teamwork	100	15 × each role goal met (A7) + 20 if no `fail` check at launch + 20 if the review was completed (solo: review panel opened and confirmed; team: all four sign-offs)

If outcome is `launch_failure` or `orbit_failure`, Objectives, Science and Operations are 0. Efficiency and Teamwork still count, so a good plan with bad luck still earns some points.

Stars: ★ ≥ 450 · ★★ ≥ 650 · ★★★ ≥ 850. **Outcome class:** `success` = all required groups satisfied AND achievedYears ≥ minimum AND scienceReturned ≥ 70% of target; `partial` = reached orbit and scienceReturned > 0; `failure` = otherwise. **Rank title:** under 450 "Trainee", 450+ "Mission Specialist", 650+ "Flight Engineer", 850+ "Flight Director".

Report explanations ("Why"): the core produces a list of `Insight { kind: 'good'|'bad', role, text, conceptId }` from rules. At least these rules: launch margin, power margin, battery, data return, lost data, pointing degraded, lifetime achieved vs planned, budget reserve, each bad event, each good event choice, missing required instrument, orbit fit < 1, debris rule. **Try next:** up to 3 concrete suggestions from the worst insights (e.g. "Your X-band link returned only 41% of the data. Try the commercial ground network or a high-gain antenna.").

A13. Online rooms (Firebase Realtime Database)

Why Firebase: the game stays a static site on GitHub Pages, and Firebase RTDB gives realtime sync, presence and anonymous sign-in with no server to run. The free Spark plan covers this event. Anonymous auth is invisible to players: no sign-up, which satisfies the "no registration" rule.

Room code: 5 characters from `ABCDEFGHJKMNPQRSTUVWXYZ23456789` (no 0/O/1/I/L). Join link: `https://<github-user>.github.io/delta-v-tycoon/#/join/ABCDE`. Max 4 connected players.

Database layout (`rooms/{code}`):

```
rooms/{code}/
  meta:      { code, hostUid, createdAt, scenarioId, difficulty, seed, phase, teamName, version: 1, opsSpeed?,
      opsPaused? }
  players/{uid}: { uid, name, characterId, connected, joinedAt, lastSeen }
  roles/{role}:  uid | null                    role = planner|engineer|science|comms
  council:   { split: { planner, engineer, science, comms }, agreed: { [uid]: true } }
  requests/{id}: { id, fromRole, toRole, amountM, reason, status: pending|accepted|declined, createdAt }
  design/planner:  { orbitId, lifetimeYears, launchVehicleId, deltaVMarginPct }
  design/engineer: { busId, solarArrayId, solarArrayW, rtgCount, batteryId, batteryWh, computerId, adcsId,
      propulsionId }
  design/science:  { instrumentIds: string[] }
  design/comms:    { radioId, hga, groundNetworkId }
  signoffs/{role}: boolean
  reviewDone: boolean
  decisions/{eventIndex}: { optionId, byUid, at }
  suggestions/{eventIndex}/{uid}: optionId
  chat/{id}: { uid, name, text (≤200 chars), at }
  result: { score, stars, outcome, designCode, at }
leaderboard/{scenarioId}/{id}: { teamName, difficulty, seed, score, stars, outcome, designCode, at }
```

Rules of play (enforced in the client; security rules in Programmer A's brief):

- The host creates the room, picks scenario and difficulty, and moves `phase` forward. Seed = random 32-bit integer at creation (or taken from a challenge code).
- Players claim roles with a transaction (first claim wins). Unclaimed roles can be claimed by anyone; one player may hold several roles. The host can release any role.
- Each role writes only its own `design/{role}` subtree. Every design write is a multi-path update that also sets all four `signoffs/*` to false.
- Operations: every client runs the same deterministic simulation locally. At each event the sim waits until `decisions/{eventIndex}` exists. The owner of the event's role writes it; when the timer expires, any client writes the default (first write wins).

- At the end, the host writes `result`. Optional (P2): post to `leaderboard`.
- Presence: `players/{uid}/connected` goes false through `onDisconnect`. If the host is gone > 30 s, the earliest-joined connected player takes over host (P1).

Same-screen and solo: `LocalRoom` implements the same `RoomApi` in memory. The single local player owns all roles. The UI is identical; only the adapter differs. This is also the fallback if the network fails.

Codes:

- **Design share code** = `lz-string.compressToEncodedURIComponent(JSON.stringify({ v:1, scenarioId, difficulty, seed, teamName, design, decisions }))`. Link `#/design/<code>`.
- **Challenge code** = `DVT-<scenarioId>-<difficulty>-<seed in base36>`, e.g. `DVT-storm_watch-engineer-k3x9q1`. Entering it fixes the scenario, difficulty and event seed for fair team-vs-team competition.

A14. Tech stack, repository and code contracts

Stack: Vite + React 18 + TypeScript (strict). State: Zustand. Network: Firebase JS SDK (modular, v10+: app, auth, database). Charts: Recharts. Share codes: lz-string. Tests: Vitest. Lint: ESLint + Prettier defaults. Deploy: GitHub Actions → GitHub Pages. Fonts (self-hosted via npm, SIL Open Font License): `@fontsource/press-start-2p` (titles), `@fontsource/vt323` (body text). Node 20 LTS.

Repository: `github.com/<lead-account>/delta-v-tycoon` — **public**. Vite `base: '/delta-v-tycoon/'`.

```
delta-v-tycoon/
  index.html  package.json  vite.config.ts  tsconfig.json  .eslintrc.cjs
  README.md            (L)   CREDITS.md (E + L)   LICENSE (MIT, L)
  database.rules.json  (A)
  .github/workflows/deploy.yml (B)
  docs/GDD.md (L)  docs/TEST_PLAN.md (Q)  docs/TEST_REPORT.md (Q)  docs/SOURCES.md (L)
  scripts/make-placeholders.mjs (B)   scripts/balance.ts (A)   scripts/check-assets.mjs (Q)
  public/assets/...    (C, E)  — exact paths in A15
  src/
    main.tsx  App.tsx  router.tsx         (B)
    assets.ts                             (B)  id → path map
    data/*.json                           (L)  content
    data/index.ts                         (A)  typed loader + validation
    core/types.ts rng.ts catalog.ts budgets.ts checks.ts sim.ts events.ts score.ts insights.ts share.ts index.ts
      (A)
    core/__tests__/*.test.ts              (A)
    net/roomApi.ts localRoom.ts firebaseRoom.ts firebase.ts roomCodes.ts leaderboard.ts   (A)
    state/store.ts                        (B)
    ui/screens/*.tsx ui/components/*.tsx ui/styles/*.css   (B)
    scenes/SpacecraftPreview.tsx OrbitView.tsx LaunchScene.tsx   (B)
    audio/sound.ts                        (B)
```

Folder ownership = merge safety. Only the owner edits a folder. If you need a change in someone else's folder, ask them on Discord. Everyone commits small and often: `git pull --rebase` → commit → push. Artists may upload through the GitHub web UI into `public/assets/...`. Commit messages: `[A] sim: add events`, `[C] portraits: nour`, etc.

A14.1 `src/core/types.ts` — the contract (Programmer A writes this file first, by H1:30)

```
export type Role = 'planner' | 'engineer' | 'science' | 'comms';
export const ROLES: Role[] = ['planner', 'engineer', 'science', 'comms'];
export type OrbitId = 'leo' | 'sso' | 'geo' | 'lunar' | 'l2' | 'mars';
export type Destination = 'LEO' | 'SSO' | 'GTO' | 'TLI' | 'L2' | 'TMI';
export type GroundNetworkId = 'campus' | 'commercial' | 'deepspace';
export type DifficultyId = 'cadet' | 'engineer' | 'director';
export type ScenarioId = 'nile_watch' | 'storm_watch' | 'ice_hunter' | 'red_relay' | 'first_light' | 'sandbox';
export type Tier = 'basic' | 'advanced';
export type Severity = 'pass' | 'warn' | 'fail';
export type Phase = 'lobby' | 'council' | 'design' | 'launch' | 'ops' | 'report';
export type Outcome = 'launch_failure' | 'orbit_failure' | 'power_failure' | 'ended_early' | 'completed';
export type OutcomeClass = 'success' | 'partial' | 'failure';
```

```
interface Item { id: string; name: string; tier: Tier; description: string }

export interface Orbit {
  id: OrbitId; name: string; destination: Destination; altitudeKm: number; periodMin: number;
  sunFraction: number; eclipseMin: number; solarFactor: number;
  insertionDvMs: number; stationKeepingDvMsPerYear: number; disposalDvMs: number;
  naturalLifetimeYears: number; commsFactor: number; contactHours: Record<GroundNetworkId, number>;
  lightDelaySec: number; radiation: number; opsCostMPerYear: number; cruiseMonths: number;
  magnetorquersWork: boolean; isNearEarth: boolean; debrisRule: boolean;
  description: string; pros: string[]; cons: string[];
}
export interface LaunchVehicle extends Item {
  classNote: string; capacityKg: Partial<Record<Destination, number>>; costM: number;
  costPerKgM?: number; minCostM?: number; reliability: number; scheduleIntervalMonths?: number;
}
export interface Bus extends Item { massKg: number; costM: number; baseLoadW: number; instrumentSlots: number;
      maxMassKg: number; hasSpareWheel: boolean }
export interface SolarArray extends Item { wPerKg: number; costMPerW: number; degradationPerYear: number }
export interface Rtg extends Item { massKg: number; powerW: number; costM: number; decayPerYear: number }
export interface Battery extends Item { whPerKg: number; costMPerWh: number; depthOfDischarge: number }
export interface Computer extends Item { massKg: number; powerW: number; costM: number; storageGbit: number;
      compression: number; autonomy: boolean; radHard: boolean }
export interface Adcs extends Item { massKg: number; powerW: number; costM: number; pointingDeg: number;
      nearEarthOnly: boolean }
export interface Propulsion extends Item { dryMassKg: number; ispS: number; costM: number; powerW: number;
      maxPropellantKg?: number; electric: boolean }
export interface Radio extends Item { massKg: number; powerW: number; costM: number; baseRateMbps: number;
      maxRateMbps: number; nearEarthOnly: boolean; rainFade: boolean }
export interface Hga extends Item { massKg: number; costM: number; gain: number; maxPointingDeg: number }
export interface GroundNetwork { id: GroundNetworkId; name: string; gain: number; costMPerYear: number;
      description: string }
export interface Instrument extends Item {
  massKg: number; powerW: number; costM: number; dataGbitPerDay: number; pointingReqDeg: number;
  tags: string[]; baseScience: number; devMonths: number; whatItMeasures: string; special?: 'particle_warning';
}
export interface Components {
  buses: Bus[]; solarArrays: SolarArray[]; rtg: Rtg; batteries: Battery[]; computers: Computer[];
  adcs: Adcs[]; propulsion: Propulsion[]; radios: Radio[]; hga: Hga; groundNetworks: GroundNetwork[];
}
export interface Scenario {
  id: ScenarioId; name: string; tagline: string; defaultDifficulty: DifficultyId; briefing: string;
  requiredTagGroups: string[][]; orbitFit: Partial<Record<OrbitId, number>>; minLifetimeYears: number;
  budgetM: number; launchWindowMonths: number; hardLaunchWindow: boolean; scienceTarget: number;
  tagWeights: Record<string, number>; defaultTagWeight: number; eventPool: string[]; eventCount: number;
  recommendedOrbit: OrbitId; learningGoals: string[]; userSegment: string; twins: MissionTwin[];
}
export interface MissionTwin { name: string; agency: string; launchYear: number; orbit: OrbitId; keyTags:
      string[]; fact: string }
export interface Difficulty { id: DifficultyId; name: string; budgetMult: number; eventMult: number;
      launchFailures: boolean; advancedUnlocked: boolean; hints: boolean; decisionTimerSec: number; description:
      string }

export type EffectType = 'loseScienceDays' | 'scienceMult' | 'powerMult' | 'contactLossDays' | 'costM' |
      'deltaVUse'
  | 'scienceBonusPct' | 'pointingMult' | 'lifetimeYearsDelta' | 'dataMult' | 'endMission' | 'downgradeGround'
  | 'immunity' | 'batteryMult' | 'relayMult';
export interface EventEffect { type: EffectType; value?: number; days?: number; reason?: string; eventId?: string
      }
export interface EventRisk {
  chance: number;                      // base chance 0..1
  chanceRule?: 'power_margin_after_hit' | 'battery_below_required';  // dynamic chance (A8.8)
  chanceIfRule?: number;               // chance used when the rule is true
  effects: EventEffect[];
  reductions?: { ifInstrument?: string; ifComputer?: string; mult: number }[];
}
export interface EventOption {
  id: string; label: string; description: string; effects: EventEffect[]; risk?: EventRisk; good: boolean;
  requires?: { propulsion?: boolean; nearEarth?: boolean; busSpareWheel?: boolean; reservePctOfBudget?: number;
      autonomyIfDelayOverSec?: number };
  explanation: string; badExplanation?: string;
}
export interface GameEvent {
  id: string; title: string; ownerRole: Role; description: string; conceptId: string;
  conditions?: { orbits?: OrbitId[]; adcsIn?: string[]; needsEclipse?: boolean };
  weightIfRadio?: { radioId: string; weight: number };
  autoEffects?: EventEffect[]; autoHandledByComputer?: string; autoHandledText?: string;
  options: EventOption[]; defaultOptionId: string; defaultOptionIdIfNoPropulsion?: string;
}

export interface DesignPlanner { orbitId: OrbitId; lifetimeYears: number; launchVehicleId: string;
      deltaVMarginPct: number }
export interface DesignEngineer { busId: string; solarArrayId: string | 'none'; solarArrayW: number; rtgCount:
      number; batteryId: string; batteryWh: number; computerId: string; adcsId: string; propulsionId: string }
```

```
export interface DesignScience { instrumentIds: string[] }
export interface DesignComms { radioId: string; hga: boolean; groundNetworkId: GroundNetworkId }
export interface Design { planner: DesignPlanner; engineer: DesignEngineer; science: DesignScience; comms:
      DesignComms }
export type BudgetSplit = Record<Role, number>;   // percents, sum 100

export interface Line { label: string; value: number; role: Role }
export interface Budgets {
  mass: { dryKg: number; propellantKg: number; tankKg: number; totalKg: number; busMaxKg: number;
      launchCapacityKg: number | null; launchMarginPct: number | null; lines: Line[] };
  power: { generationW: number; loadW: number; marginPct: number; rtgW: number; batteryWh: number;
      batteryRequiredWh: number; batteryOk: boolean; eclipseMin: number; lines: Line[] };
  data: { generatedGbitPerDay: number; downlinkGbitPerDay: number; rateMbps: number; contactHoursPerDay: number;
      returnPct: number; storageGbit: number; hgaEffective: boolean };
  deltaV: { requiredMs: number; withMarginMs: number; capabilityMs: number; ispS: number; feasible: boolean };
  cost: { byRole: Record<Role, number>; allocationM: Record<Role, number>; launchM: number; opsM: number; delayM:
      number; totalM: number; budgetM: number; reservePct: number; lines: Line[] };
  time: { devMonths: number; launchMonth: number; windowMonths: number; delayMonths: number; transferMonths:
      number; lifetimeYears: number };
  pointing: { adcsDeg: number; perInstrument: { id: string; reqDeg: number; quality: number }[] };
}
export interface Check { id: string; severity: Severity; role: Role; title: string; message: string; fixHint:
      string; conceptId: string }
export interface SimInput { scenarioId: ScenarioId; difficulty: DifficultyId; seed: number; design: Design;
      split: BudgetSplit }
export interface PendingEvent { index: number; day: number; event: GameEvent; options: (EventOption & { disabled:
      boolean; disabledReason?: string })[]; autoText?: string }
export interface Sample { day: number; generationW: number; loadW: number; duty: number; stored: number;
      dataGeneratedCum: number; dataReturnedCum: number; scienceCum: number; costCum: number }
export interface EventLogEntry { index: number; day: number; eventId: string; title: string; optionId: string |
      null; byRole: Role; bad: boolean; text: string }
export interface SimResult {
  outcome: Outcome; outcomeReason: string; endDay: number; plannedDays: number; achievedYears: number;
  dataGenerated: number; dataReturned: number; dataLost: number; scienceGenerated: number; scienceReturned:
      number;
  extraCostM: number; finalTotalCostM: number; samples: Sample[]; events: EventLogEntry[];
}
export interface Simulation {
  readonly day: number; readonly pendingEvent: PendingEvent | null; readonly done: boolean;
  step(days: number): void;                // advances; stops early at an event or the end
  decide(optionId: string): void;          // resolves pendingEvent
  current(): Sample;                       // latest values for the live dashboard
  result(): SimResult;                     // valid when done
}
export interface Insight { kind: 'good' | 'bad'; role: Role; text: string; conceptId: string }
export interface ScoreReport {
  total: number; stars: 0 | 1 | 2 | 3; outcomeClass: OutcomeClass; rankTitle: string;
  parts: { objectives: number; science: number; efficiency: number; operations: number; teamwork: number };
  roleGoals: Record<Role, { met: boolean; text: string }>;
  insights: Insight[]; tryNext: string[];
}
export interface TeamContext { reviewDone: boolean; failChecksAtLaunch: number }
export interface Maturity { cml: 2 | 3 | 4; name: 'Initial Feasibility' | 'Trade Space' | 'Point Design';
      greenPct: number; green: number; yellow: number; red: number }
export type SweepParam = 'lifetimeYears' | 'solarArrayW' | 'batteryWh' | 'deltaVMarginPct' | 'busId' | 'radioId'
      | 'groundNetworkId' | 'launchVehicleId' | 'adcsId';
export interface SweepPoint { value: number | string; label: string; scienceEst: number; scienceEstPctOfTarget:
      number; totalCostM: number; launchMarginPct: number | null; powerMarginPct: number; returnPct: number;
      failCount: number; isCurrent: boolean }
```

A14.2 Core API (`src/core/index.ts`, pure functions, no React, no Firebase)

```
export function getCatalog(): Catalog;      // { orbits, launchVehicles, components, instruments, scenarios,
      difficulties, events, glossary, checksText, dialog, characters, constants }
export function defaultDesign(scenarioId: ScenarioId, difficulty: DifficultyId): Design;   // a valid, slightly
      imperfect starter design
export const DEFAULT_SPLIT: BudgetSplit;    // { planner: 35, engineer: 30, science: 25, comms: 10 }
export function computeBudgets(design: Design, scenarioId: ScenarioId, difficulty: DifficultyId, split:
      BudgetSplit): Budgets;
export function runChecks(design: Design, scenarioId: ScenarioId, difficulty: DifficultyId, split: BudgetSplit):
      Check[];
export function isUnlocked(itemId: string, difficulty: DifficultyId, scenarioId: ScenarioId): boolean;
   // true if: item tier is basic, OR difficulty.advancedUnlocked, OR scenario is sandbox,
   //          OR the item is an instrument whose tags satisfy one of the scenario's required groups
```

```
export function createSimulation(input: SimInput): Simulation;
export function simulateAll(input: SimInput, decisions: Record<number, string>): SimResult;   // headless replay
export function scoreMission(input: SimInput, result: SimResult, team: TeamContext): ScoreReport;
export function encodeDesign(p: SharePayload): string;  export function decodeDesign(code: string): SharePayload
      | null;
export function encodeChallenge(s: ScenarioId, d: DifficultyId, seed: number): string;  export function
      decodeChallenge(code: string): { scenarioId: ScenarioId; difficulty: DifficultyId; seed: number } | null;
export function newSeed(): number;
export function conceptMaturity(checks: Check[]): Maturity;                                    // F-35
export function findMissionTwin(design: Design, scenarioId: ScenarioId): MissionTwin | null;  // F-33
export function estimateScience(design: Design, scenarioId: ScenarioId, difficulty: DifficultyId, split:
      BudgetSplit): number;   // no events
export function tradeSweep(design: Design, scenarioId: ScenarioId, difficulty: DifficultyId, split: BudgetSplit,
      param: SweepParam): SweepPoint[];  // F-34
// SharePayload = { v: 1; scenarioId; difficulty; seed; teamName: string; design: Design; split: BudgetSplit;
      decisions: Record<number, string> }
```

A14.3 Room API (src/net/roomApi.ts)

```
export interface PlayerState { uid: string; name: string; characterId: string; connected: boolean; joinedAt:
      number; lastSeen: number }
export interface FundsRequest { id: string; fromRole: Role; toRole: Role; amountM: number; reason: string;
      status: 'pending' | 'accepted' | 'declined'; createdAt: number }
export interface ChatMessage { id: string; uid: string; name: string; text: string; at: number }
export interface RoomMeta { code: string | null; hostUid: string; createdAt: number; scenarioId: ScenarioId;
      difficulty: DifficultyId; seed: number; phase: Phase; teamName: string; version: 1; opsSpeed?: 1 | 2 | 4 |
      8; opsPaused?: boolean }
export interface RoomState {
  meta: RoomMeta; players: Record<string, PlayerState>; roles: Record<Role, string | null>;
  council: { split: BudgetSplit; agreed: Record<string, boolean> }; requests: FundsRequest[];
  design: Design; signoffs: Record<Role, boolean>; reviewDone: boolean;
  decisions: Record<number, { optionId: string; byUid: string; at: number }>;
  suggestions: Record<number, Record<string, string>>; chat: ChatMessage[];
  result: { score: number; stars: number; outcome: string; designCode: string; at: number } | null;
}
export interface RoomApi {
  readonly mode: 'local' | 'online';
  readonly myUid: string;
  subscribe(cb: (s: RoomState) => void): () => void;   // called immediately with current state, then on every
      change
  getState(): RoomState;
  setMeta(patch: Partial<Pick<RoomMeta, 'scenarioId' | 'difficulty' | 'phase' | 'teamName' | 'seed' | 'opsSpeed'
      | 'opsPaused'>>): Promise<void>;  // host only
  updateMe(patch: Partial<Pick<PlayerState, 'name' | 'characterId'>>): Promise<void>;
  claimRole(role: Role): Promise<boolean>;
  releaseRole(role: Role): Promise<void>;
  setSplit(split: BudgetSplit): Promise<void>;          // clears all agreed
  setAgreed(agreed: boolean): Promise<void>;
  requestFunds(fromRole: Role, toRole: Role, amountM: number, reason: string): Promise<void>;
  answerRequest(id: string, accept: boolean): Promise<void>;   // accept moves split points
  updateDesign<R extends Role>(role: R, patch: Partial<Design[R]>): Promise<void>;   // also clears all signoffs
  setSignoff(role: Role, value: boolean): Promise<void>;
  setReviewDone(value: boolean): Promise<void>;
  submitDecision(eventIndex: number, optionId: string): Promise<void>;   // first write wins
  suggest(eventIndex: number, optionId: string): Promise<void>;
  sendChat(text: string): Promise<void>;
  setResult(r: NonNullable<RoomState['result']>): Promise<void>;
  leave(): Promise<void>;
}
export function createLocalRoom(opts: { name: string; characterId: string; scenarioId: ScenarioId; difficulty:
      DifficultyId; seed?: number }): RoomApi;
export function createOnlineRoom(opts: { name: string; characterId: string; scenarioId: ScenarioId; difficulty:
      DifficultyId; seed?: number }): Promise<RoomApi>;   // generates code
export function joinOnlineRoom(code: string, opts: { name: string; characterId: string }): Promise<RoomApi>;   //
      throws 'ROOM_NOT_FOUND' | 'ROOM_FULL' | 'NETWORK'
export function myRoles(s: RoomState, uid: string): Role[];
export function isHost(s: RoomState, uid: string): boolean;
```

A14.4 UI store (src/state/store.ts, Programmer B) — shape all screens use

```
interface GameStore {
  room: RoomApi | null; state: RoomState | null;
  budgets: Budgets | null; checks: Check[];                // recomputed whenever state.design/split changes
  sim: Simulation | null; simSpeed: 1 | 2 | 4 | 8; paused: boolean;
  score: ScoreReport | null; attempts: { at: number; score: number; designCode: string }[];
  settings: { volume: number; muted: boolean; textScale: 1 | 1.25 | 1.5; reducedMotion: boolean };
  startSolo(o): void; createRoom(o): Promise<void>; joinRoom(code, o): Promise<void>; leaveRoom(): Promise<void>;
}
```

A15. Art direction and the complete asset manifest

Style: clean, readable pixel art; NASA-mission-control mood with warm Egyptian sunlight accents. Light comes from the **top-left**. 1-pixel dark outline (#181425) on characters and objects, selective (lighter outline colour where an edge faces the light). Maximum 3 shades per material + 1 highlight. No anti-aliasing, no blur, no gradients except dithering. Every PNG is drawn at **1× native size** and exported as PNG-32 with transparency; the game scales it ×2, ×3 or ×4 with nearest-neighbour filtering. **Never** export upscaled images.

Palette — ENDESGA 32 (use only these 32 colours):

```
#be4a2f #d77643 #ead4aa #e4a672 #b86f50 #733e39 #3e2731 #a22633
#e43b44 #f77622 #feae34 #fee761 #63c74d #3e8948 #265c42 #193c3e
#124e89 #0099db #2ce8f5 #ffffff #c0cbdc #8b9bb4 #5a6988 #3a4466
#262b44 #181425 #ff0044 #68386c #b55088 #f6757a #e8b796 #c28569
```

UI colour mapping (CSS variables, Programmer B): background #181425 · panel #262b44 · panel border #3a4466 · text #ffffff · muted text #c0cbdc / #8b9bb4 · accent gold #feae34 · primary blue #0099db · pass #63c74d · warn #f77622 · fail #e43b44 · role planner #feae34 · engineer #f77622 · science #63c74d · comms #2ce8f5. Status is never shown by colour alone: pass = check mark, warn = triangle "!", fail = cross.

Skin tones from the palette: light #e8b796 (shade #c28569), medium #c28569 (shade #b86f50), brown #b86f50 (shade #733e39), dark #733e39 (shade #3e2731). Never use pure black for skin shading.

A15.1 Crew roster (`src/data/characters.json`)

id	name	from	game role	playable	appearance summary
nour	Nour El-Sayed	Egypt	planner	yes	medium skin; navy hijab (#124e89) with gold edge, neatly wrapped; calm confident smile; ~35
lucas	Lucas Moreau	France	planner	yes	light skin; short wavy brown hair; light stubble; round glasses; ~40
diaa	Diaa El Awdi	Egypt	planner	yes	a real person (appears with his permission); light-olive skin; short salt-and-pepper hair combed back (grey sides, darker top); dark eyebrows; wide warm smile; clean-shaven; signature navy suit, light-blue shirt, silver tie, gold watch; ~50s
omar	Omar Farouk	Egypt	engineer	yes	brown skin; short black curly hair; short beard; rolled sleeves; ~30
elena	Elena Petrova	Russia	engineer	yes	light skin with freckles; blonde hair in a bun; focused look; ~45
amara	Amara Okafor	Nigeria	science	yes	dark skin; natural coily hair in two puffs; gold hoop earrings; holds tablet; ~32
hiroshi	Hiroshi Tanaka	Japan	science	yes	light skin; straight black hair with grey streaks; field notebook; ~55
kwame	Kwame Mensah	Ghana	comms	yes	dark skin; short fade haircut; headset with mic; ~28
layla	Layla Al-Mansouri	UAE	comms	yes	medium skin; long dark ponytail; headset; bright smile; ~27
hassan	Dr. Hassan Nasr	Egypt	flight director (NPC, tutorial guide)	no	brown skin; short grey beard; glasses; mission vest; warm smile; ~60
priya	Priya Nair	India	ground station operator (NPC, comms tips)	no	brown skin; long dark braid; headset; uses a wheelchair; ~34

Every crew member wears a white flight suit (#ffffff / #c0cbdc / #8b9bb4) with a shoulder stripe and chest patch in their **role colour**. Exception: Diaa El Awdi wears his signature navy suit (with a gold Commander pin on the lapel) in his portraits and idle sprite; his suit sprite is the standard spacesuit. NPCs wear the mission control vest in #3a4466 with a gold badge. All characters are adults.

A15.2 Asset manifest (exact paths under `public/assets/`)

Programmer B generates a coloured placeholder for every row at H1 (`scripts/make-placeholders.mjs`). Artists overwrite the same path with the final art: the game picks it up with no code change. Character art = **C**, everything else = **E**.

Path	Size (px, 1×)	Frames	Owner	Content
characters/portrait_{id}_{expr}.png	64×64	1	C	for each of 11 ids × expr ∈ neutral, happy, worried (33 files); head and shoulders, transparent background

Path	Size (px, 1×)	Frames	Owner	Content
characters/sprite_{id}_idle.png	128×48	4 × (32×48)	C	11 files; standing idle (breathing/blink loop), flight suit, no helmet; facing right-front
characters/sprite_{id}_suit.png	64×48	2 × (32×48)	C	9 playable only; full spacesuit with helmet visor up (face visible); frame 2 = waving
bg/bg_title.png	480×270	1	E	Earth's limb, stars, a rocket climbing, logo space in upper third left clear
bg/bg_mission_control.png	480×270	1	E	mission control room, big wall screen, consoles; lower 90 px calm for UI
bg/bg_clean_room.png	480×270	1	E	spacecraft assembly clean room; centre area empty (spacecraft preview drawn there)
bg/bg_launch_pad.png	480×270	1	E	coastal desert launch site at dawn; launch tower right of centre; rocket drawn by code at x=240
bg/bg_space_tile.png	64×64	1	E	seamless tiling starfield
planets/planet_earth.png	128×128	1	E	Earth with Africa/Nile visible
planets/planet_moon.png	96×96	1	E	Moon, south pole craters visible
planets/planet_mars.png	112×112	1	E	Mars
planets/planet_sun.png	64×64	1	E	Sun disc with glow (dithered)
lv/lv_spark.png	16×64	1	E	small rocket
lv/lv_arrow.png	24×96	1	E	light-medium rocket
lv/lv_workhorse.png	24×128	1	E	medium rocket (also used for rideshare)
lv/lv_titan.png	48×128	1	E	heavy rocket with side boosters
fx/fx_flame.png	64×24	4 × (16×24)	E	engine flame loop (scaled to rocket width by code)
fx/fx_smoke.png	192×32	6 × (32×32)	E	launch smoke puff
fx/fx_explosion.png	512×64	8 × (64×64)	E	launch failure explosion (cartoonish, not graphic)
sc/bus_cube.png · bus_small.png · bus_medium.png · bus_large.png	16×16 · 32×32 · 48×48 · 64×64	1	E	spacecraft body, side view, gold foil + panels
sc/sa_si_panel.png · sc/sa_gaas_panel.png	16×24	1	E	one solar panel (silicon = blue #124e89, GaAs = dark #262b44 with violet glint)
sc/rtg.png	8×16	1	E	finned cylinder
sc/hga.png	24×24	1	E	white dish antenna
sc/antenna_lga.png	8×8	1	E	small stub antenna
sc/thruster.png	8×8	1	E	nozzle
sc/sc_marker.png	8×8	1	E	tiny spacecraft for the orbit view
items/{id}.png	32×32	1	E	one icon per catalog id (list A15.3)
icons/{id}.png	16×16	1	E	res_mass res_power res_budget res_data res_comms res_time res_deltav res_science st_pass st_warn st_fail st_info role_planner role_engineer role_science role_comms ui_star_full ui_star_empty ui_lock ui_help
events/ev_{eventId}.png	96×64	1	E	illustration per event (10 files)
scenarios/sc_{scenarioId}.png	160×90	1	E	scenario card art (6 files)
ui/ui_panel.png · ui_panel_dark.png	24×24	1	E	9-slice frame, 8 px corners
ui/ui_button.png · _hover · _pressed · _disabled	24×24	1	E	9-slice button, 8 px corners
ui/ui_tab.png · ui_tab_active.png	24×24	1	E	9-slice tab
ui/logo_deltav.png	256×64	1	E	"DELTA-V TYCOON" pixel logotype
ui/favicon.png	32×32	1	E	rocket icon
ui/og_image.png	1200×630	1	E	link preview (logo + crew lineup); may be composed from other art
audio/music_menu.mp3 · music_ops.mp3	loop	–	E	calm synth loops, CC0 or CC-BY

Path	Size (px, 1×)	Frames	Owner	Content
audio/sfx_click · sfx_hover · sfx_success · sfx_fail · sfx_warning · sfx_launch · sfx_event · sfx_signoff · sfx_coin .mp3	short	–	E	sound effects, CC0 or CC-BY

A15.3 `items/` icon ids (32×32 each, 52 files)

Buses: bus_cube bus_small bus_medium bus_large · Power: sa_si sa_gaas rtg bat_liion bat_liion_adv · Computers: obc_basic obc_adv · ADCS: adcs_mag adcs_wheels adcs_precision · Propulsion: prop_none prop_coldgas prop_mono prop_biprop prop_electric · Comms: radio_uhf radio_sband radio_xband radio_kaband hga · Ground: gs_campus gs_commercial gs_deepspace · Vehicles: lv_spark lv_rideshare lv_arrow lv_workhorse lv_titan · Orbits: orbit_leo orbit_sso orbit_geo orbit_lunar orbit_l2 orbit_mars · Instruments: cam_wide cam_hires imager_ms imager_tir radar_sar sounder_atm lightning_map spec_neutron lidar_alt magnetometer spec_atm det_particle relay_uhf tel_ir.

Spacecraft preview composition (Programmer B draws it; artists keep these anchors in mind): canvas 160×120 (1×). Bus centred at (80,60). Solar panels attach at the bus left/right edge centre; panel count per side = `clamp(ceil(solarArrayW / 500), 0, 6)`, laid side by side outward. HGA on top centre; low-gain antenna on top-right corner; RTGs on bottom-left; thruster at bottom centre; instruments drawn as their 32×32 item icons scaled to 16×16 on the front face in a row, max 6.

A16. Timeline, communication and definition of done

All times are Cairo time. H0 = Friday 1:00 PM.

When	Milestone (owner)
Before H0 (allowed)	Accounts: GitHub (all), Firebase project (A), itch.io optional (B). Install Node 20, VS Code, Aseprite/LibreSprite/Piskel. Join event Discord, choose mentor tracks. Create EMPTY public repo (L). Read this whole document.
H0–H0:30	Kickoff: Lead walks everyone through Part A. Challenge resources (released at kickoff) reviewed by Lead.
H1	Repo scaffold + placeholders + deploy workflow pushed (B). Palette + style test: 1 portrait (C) and 3 icons (E) shown to Lead.
H1:30	`src/core/types.ts` pushed (A).
H2	All `src/data/*.json` v1 pushed (L). Test plan skeleton (Q).
H4	`computeBudgets` + `runChecks` + `createLocalRoom` working (A). Title → solo → design screen with live meters (B).
H6 (7 PM)	**First public build online.** Solo: design → review → launch → ops (no events yet) → report. QA starts build testing.
H9 (10 PM)	Events + scoring + insights + Mission Twin + Concept Maturity done (A). All P0 screens incl. review meter, report, briefing with requirements (B). Online create/join working (A). Playable portraits + item icons in (C, E).
H11 (12 AM)	Online lobby, council, sign-offs, decisions working (A+B). Trade-Space Explorer, tutorial, Academy (A+B+L). All art in (C, E).
H12 (1 AM)	**Online go/no-go test (Q). Cutoff rule:** if online rooms are not stable, the title screen shows "Team Mode (same screen)" and the online buttons are hidden behind `?online=1`.
H13 (2 AM)	Balance pass (L+A), P1 polish, text polish.
H14 (3 AM)	**FEATURE FREEZE.** Bug fixes only. Tag `golden-1` on the last good build.
H14–H16 (3–5 AM)	P0 bug fixes (A, B); regression pass (Q); footage capture (Q+L). Sleep shifts begin: max 2 people asleep at a time.
H16–H21 (5–10 AM)	Video edit + slides (L, help from C/E), README + summary + sources (L). Only P0 fixes. Tag `golden-final` by 10 AM.
H22–H23 (11 AM–12 PM)	**Submit** local form, then global site (L). QA verifies every link in a private window.
H24 (1 PM)	Official deadline. Then rest; prepare for People's Choice at ~5 PM.

Mentor tracks (each member picks a different one on the event Discord): Lead → Storytelling & Video Editing · Programmer A → Software Engineering · Programmer B → Web Development · Character artist → Art & Visualization · Environment/asset artist → Game Development · QA → UI/UX Design.

Discord channels (team server or a private group): `#general` (coordination), `#bible-changes` (Lead only), `#builds` (B posts every deploy URL + version), `#bugs` (Q; mirrors GitHub Issues), `#art-drops` (C/E post previews), `#help` (blocked? post here immediately). **Status ping every 2 hours** from every member: done / doing / blocked.

Definition of done (for any task): it works in the deployed public build, not only on your laptop; it uses the exact IDs and paths of this document; it has no console errors; QA has seen it once.

A17. Reference designs (expected values — tests and QA use these)

Computed with the formulas in A9 (rounded). If the code gives different numbers, the code is wrong (or A9 was changed — check `#bible-changes`).

Scenario	Design	Total kg	Launch margin	Load W	Gen W	Power margin	Battery req Wh	Downlink Gbit/day	Return	Cost $M
nile_watch	SSO 3 yr, Spark, 10% margin, bus_small, sa_si 450 W, bat_liion 350 Wh, obc_basic, adcs_wheels, prop_mono, imager_ms + imager_tir, radio_xband, campus	185.7	7.1%	189.8	248.6	31.0%	277	216.0	100%	70.8
storm_watch	GEO 5 yr, Workhorse, 10%, bus_medium, sa_si 700 W, bat_liion 1500 Wh, obc_basic, adcs_wheels, prop_biprop, sounder_atm + imager_tir + lightning_map, radio_xband, commercial	1005.8	81.7%	474.0	589.0	24.3%	1422	1944.0	100%	219.7
ice_hunter	Lunar 1.5 yr, Arrow, 10%, bus_medium, sa_si 700 W, bat_liion 600 Wh, obc_basic, adcs_wheels, prop_biprop, spec_neutron + lidar_alt + imager_tir, radio_xband, deepspace	626.4	10.5%	299.0	357.0	19.4%	586	259.2	100%	143.8
red_relay	Mars 2 yr, Workhorse, 10%, bus_medium, sa_gaas 1800 W, bat_liion 600 Wh, obc_adv, adcs_wheels, prop_biprop, spec_atm + relay_uhf + cam_wide, radio_xband + hga, deepspace	705.8	71.8%	353.2	427.6	21.1%	589	38.9	100%	194.7
first_light	L2 5 yr, Workhorse, 10%, bus_medium, sa_gaas 750 W, bat_liion 100 Wh, obc_basic, adcs_precision, prop_mono, tel_ir, radio_kaband, deepspace	755.1	74.8%	520.7	637.5	22.4%	50	77.8	100%	487.7
(bad example)	LEO 2 yr, Spark, bus_cube, sa_si 100 W, bat 50 Wh, obc_basic, adcs_mag, prop_none, cam_wide, radio_sband, campus	21.8	92.7%	30.4	52.7	73.3%	50	3.6	18%	18.9

The bad example is deliberately cheap: it launches fine but returns only 18% of its data, and the magnetorquers blur the camera (quality 0.2). The report must explain both. **Several different designs win each scenario** (e.g. ice_hunter on Arrow at 10.5% margin vs Workhorse at 82%: cheaper but riskier). That is the "no single perfect solution" point judges look for.